

# ЛЕКЦИЯ 3

ПРОДВИНУТЫЙ CSS

# ВОПРОСЫ

1. CSS и его роль в веб-разработке
2. Блочная модель
3. Внешние, встроенные и инлайн-стили. Импорт
4. CSS селекторы
5. Специфичность
6. Каскадность
7. Наследование
8. Контекст наложения
9. Фон, граница, цвет
10. Единицы измерения
11. Псевдоэлементы
12. Подходы к созданию классов, или архитектура CSS

1

# CSS И ЕГО РОЛЬ В ВЕБ-РАЗРАБОТКЕ

me: let's rewrite the CSS  
my website:

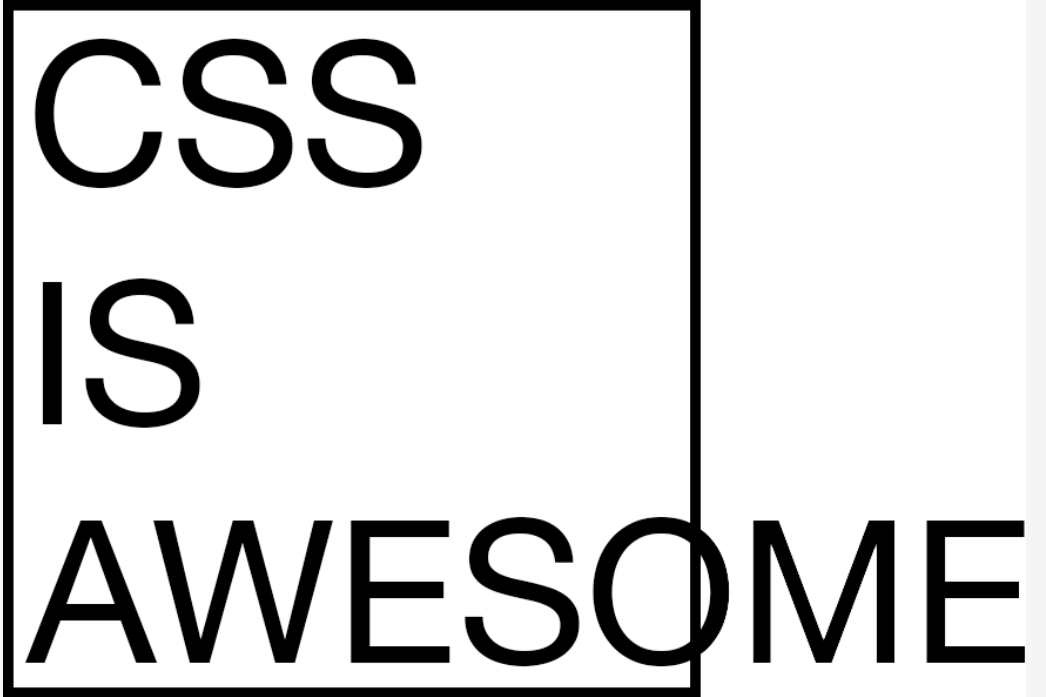


# CSS

**CSS** (Cascading Style Sheets, «каскадные таблицы стилей») — это язык стилей, с помощью которого оформляют веб-страницы.

Он определяет, как именно должны отображаться элементы HTML, включая макет, цвета, шрифты и другие визуальные аспекты. CSS используют для того, чтобы отделить содержание веб-страницы от её оформления.

За счёт этого страницы удобнее изменять и поддерживать.



CSS  
IS  
AWESOME

# CSS

**CSS**



**CSS** был разработан в 1996 году с целью отделить содержание веб-страниц от их оформления.

С тех пор CSS претерпел множество изменений и улучшений, включая появление различных версий, таких как CSS2 и CSS3, которые добавили новые возможности и улучшили поддержку различных устройств

Правило CSS – блок кода, содержащий один или несколько селекторов и одно или несколько объявлений.

## Части правила CSS

Sciencetech Easy

Selector

h1 {

color: blue;

font-size: 24px;

text-align: center;

}

Declaration block

First declaration

Second declaration

Thrid declaration

color: blue;

value1

property1

font-size: 24px;

value2

property2

text-align: center;

value3

property3

В этом CSS-правиле **селектором** является `.my-css-rule`, который находит все элементы с классом `my-css-rule` на странице.

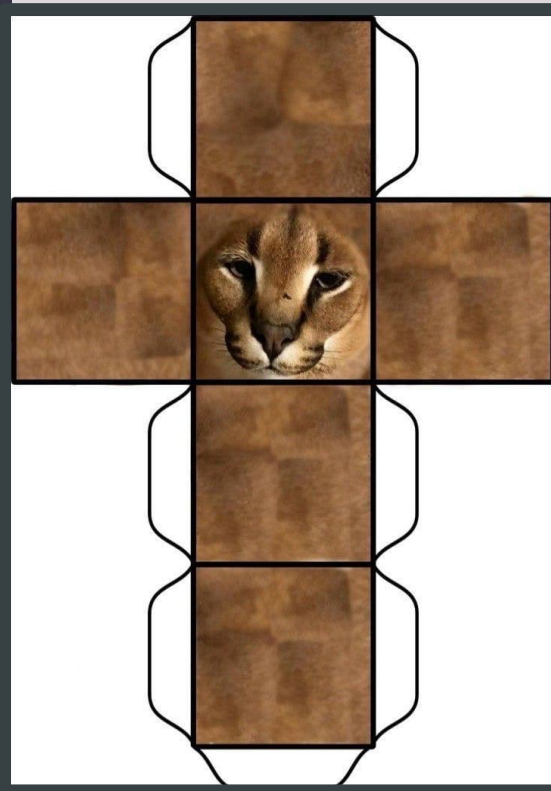
Внутри фигурных скобок находятся три **декларации**.

Объявление — это пара **свойств и значений**, которая применяет стили к элементам, подобранным селекторами.

В правиле CSS может быть сколько угодно деклараций и селекторов

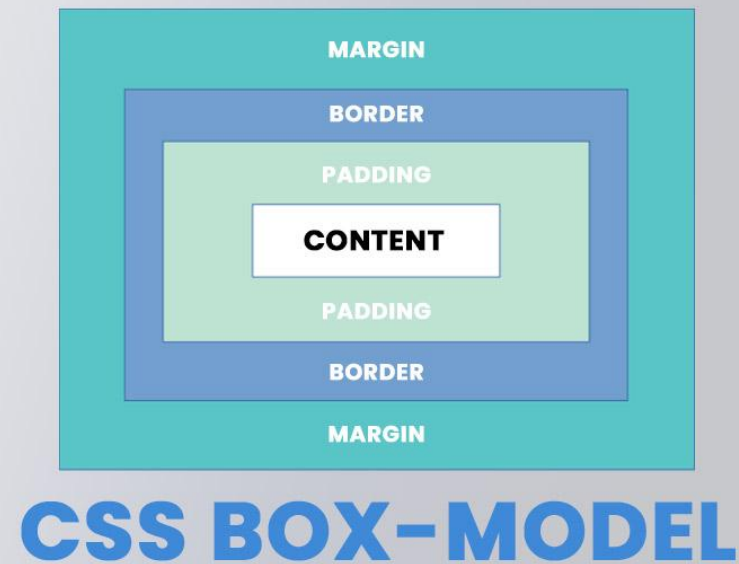
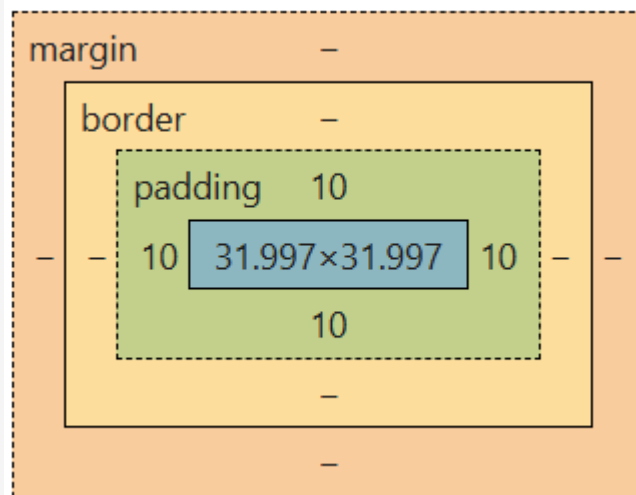
2

## БЛОЧНАЯ МОДЕЛЬ

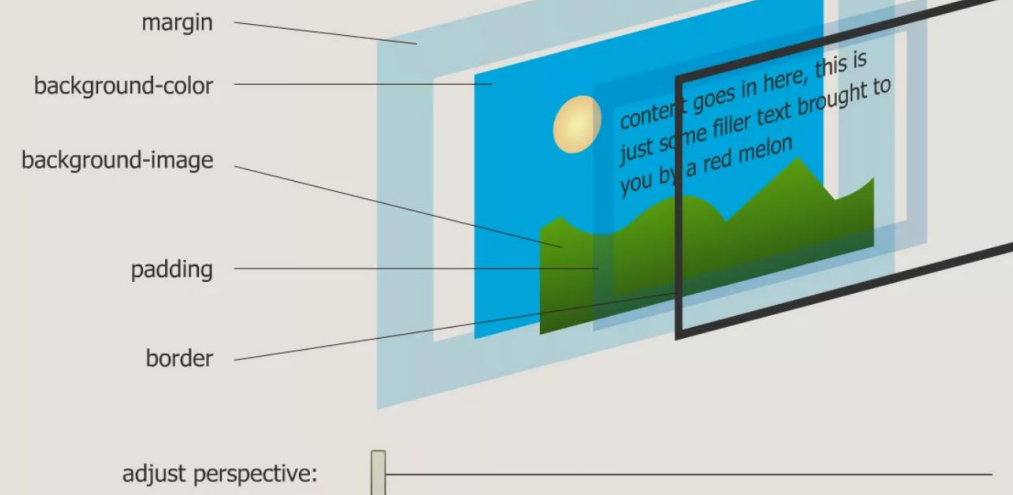


# Блочная модель

**Блочная модель**, она же box model — это алгоритм расчёта размеров каждого отдельного элемента на странице, которым браузеры пользуются при отрисовке.



## Basic CSS Box Model Demo



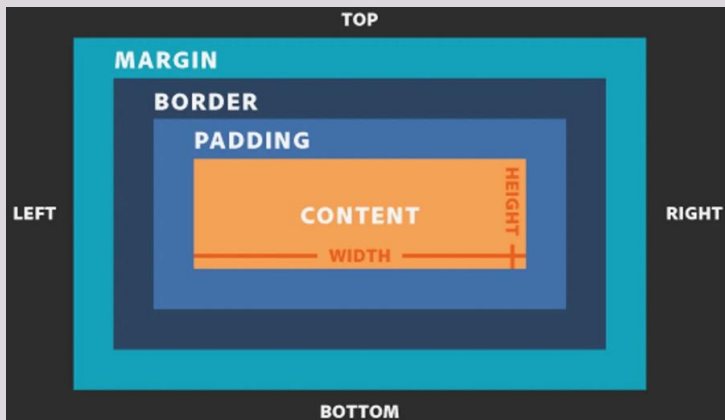


# Составляющие блочной модели

1

**Контент** – это область, где отображается текст или изображение.

Размеры контента зависят от его содержимого и заданных свойств



2

**Внутренний отступ / padding** – отступы создают пространство между содержимым и границей элемента. Они могут быть заданы отдельно для каждой из сторон или применяться ко всем сторонам одновременно.

```
.example {  
    padding: 10px; /* Все стороны: 10px */  
    padding: 10px 20px; /* Вертикально / Горизонтально: 10px  
20px */  
    padding: 5px 10px 15px 20px; /* Верх | Право | Низ | Лево:  
5px 10px 15px 20px */  
    padding-top: 10px; /* Верхний отступ */  
    padding-right: 15px; /* Правый отступ */  
}
```

# Составляющие блочной модели

## 3

**Граница** окружает элемент и может иметь различные стили, толщину и цвет

Она добавляет дополнительное пространство вокруг внутреннего отступа

```
.example {  
  border-width: 2px; /* Толщина  
    границы */  
  border-style: solid; /*Стиль*/  
  border-color: red; /*Цвет*/  
  border: 2px dashed blue; /*  
    Толщина | Стиль | Цвет */  
}
```

## 4

**Внешний отступ / margin** –создают пространство между элементами. Они также могут быть заданы индивидуально или применяться ко всем сторонам одновременно.

```
.example {  
  margin: inherit; /* Наследование значения от родителя */  
  margin-bottom: 15px; /* Нижний внешний отступ */  
  margin-left: 10px; /* Левый внешний отступ */  
  margin: 5px auto 10px; /* Верх | Авто-горизонтально | Низ:  
    5px auto 10px */  
  margin: 5px 10px 15px 20px; /* Верх | Право | Низ | Лево:  
    5px 10px 15px 20px */  
}
```

# ПРИМЕР

Если элемент должен занимать больше места, чем вложенный в него контент, то мы меняем его ширину и высоту так, как нам захочется.

Например:

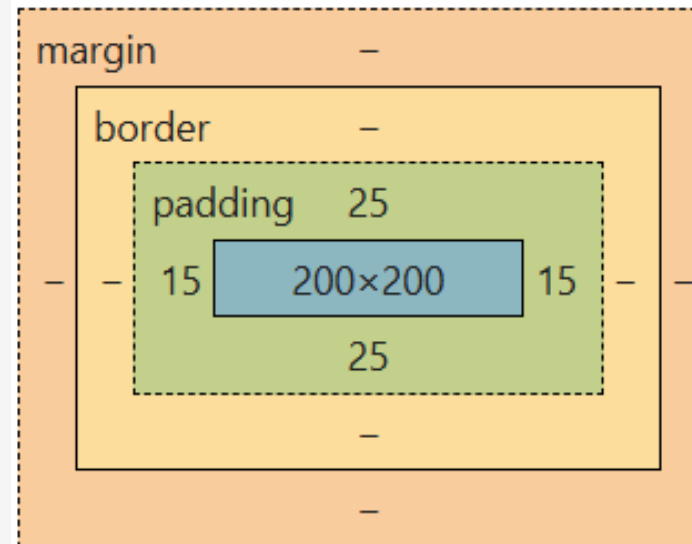
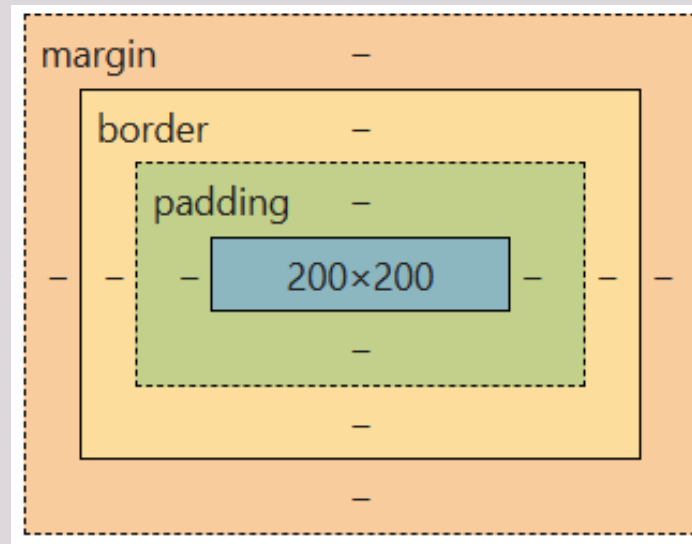
```
p {  
    width: 200px;  
    height: 200px;  
}
```

*Теперь элемент будет размером 200 на 200 пикселей.*

Добавим к стилям внутренние отступы (padding)

```
p {  
    width: 200px;  
    height: 200px;  
    padding: 25px 15px;  
}
```

*Теперь ширина блока будет равна  $200 + 15 + 15 = 230$ .  
А высота будет равна  $200 + 25 + 25 = 250$  пикселей.*



# ПРИМЕР

Пусть у элемента будет  
рамка со всех сторон

```
p {  
  width: 200px;  
  height: 200px;  
  padding: 25px 15px;  
  border: 5px solid red;  
}
```

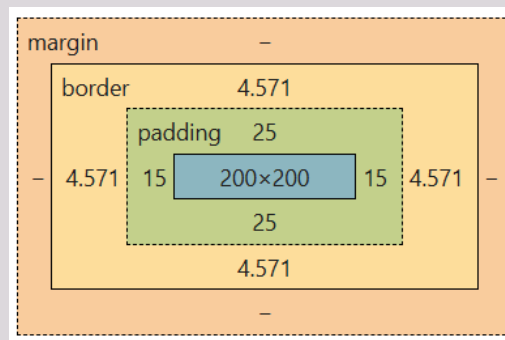
**Внешние отступы**, за которые отвечает  
свойство **margin**, не прибавляются  
непосредственно к размерам элемента,  
но влияют на то, сколько места на странице  
он занимает.

Если элементу из примера зададим внешние  
отступы, то он будет занимать больше места,  
двигая при этом своих соседей.

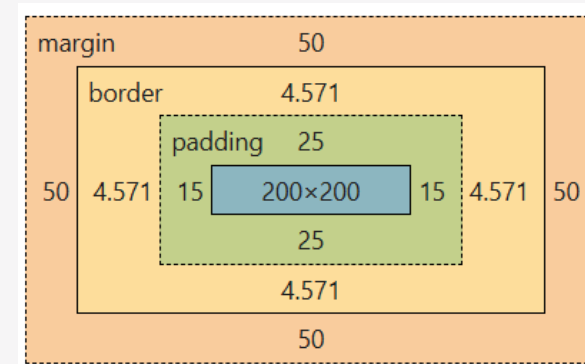
*Теперь конечные размеры элемента будут:*

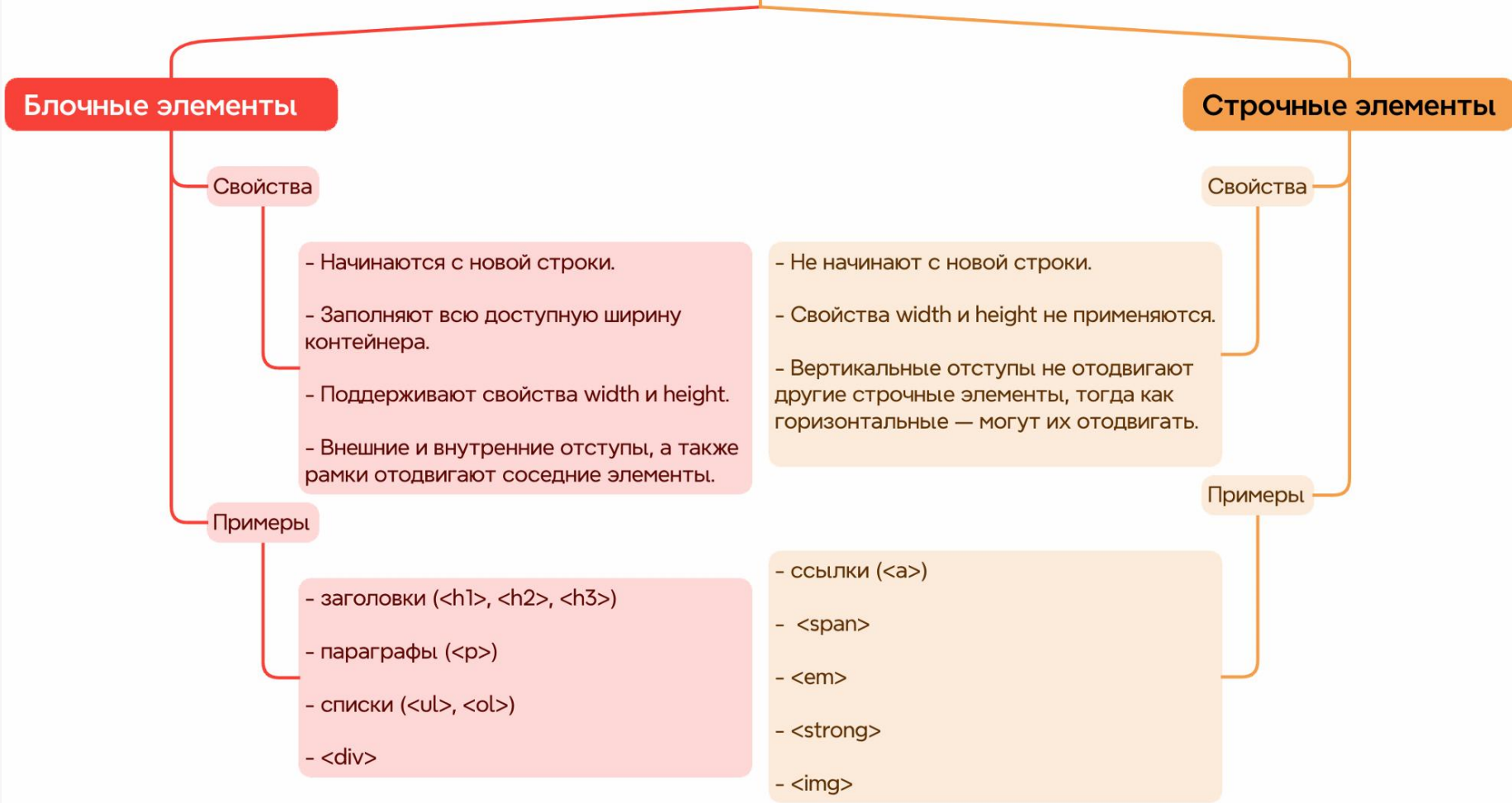
*$200 + 15 + 15 + 5 + 5 = 240$  пикселей в ширину*

*$200 + 25 + 25 + 5 + 5 = 260$  пикселей в высоту*

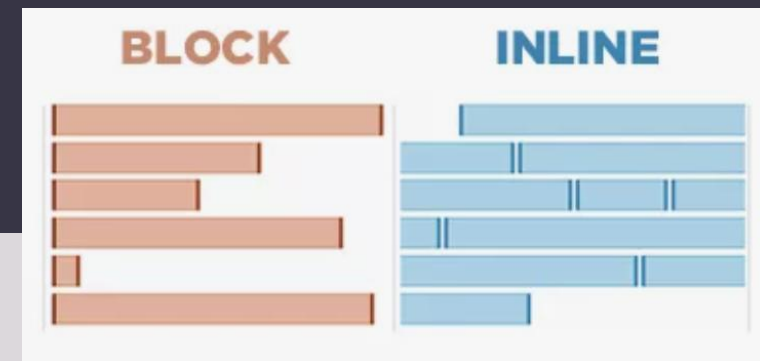


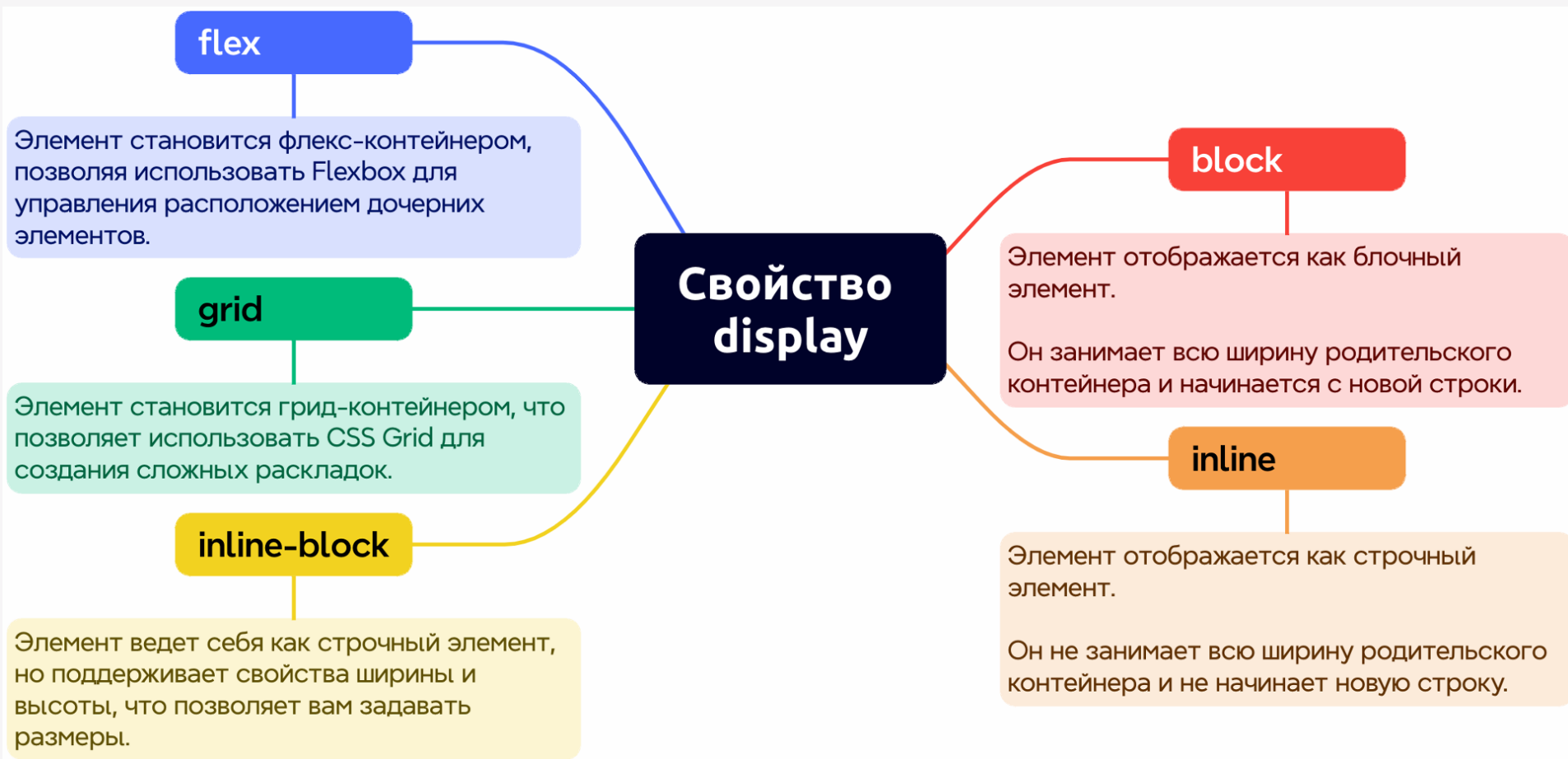
```
p {  
  width: 200px;  
  height: 200px;  
  padding: 25px 15px;  
  margin: 50px;  
  border: 5px solid red;  
}
```





# Блочные и строчные элементы

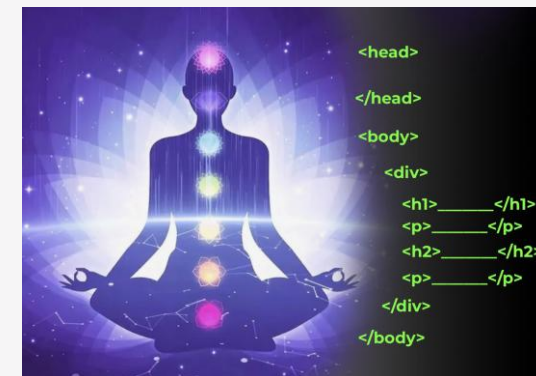




## Свойство display

Свойство display позволяет вам управлять тем, как элемент будет отображаться на странице.

Блочные элементы располагаются вертикально друг под другом, тогда как строчные элементы располагаются горизонтально в пределах строки. При добавлении новых элементов важно учитывать, как они будут влиять на поток документа.



Если вы добавите блочный элемент между двумя строчными элементами, это приведет к тому, что блочный элемент начнет новую строку, а строчные элементы будут располагаться выше и ниже него.

## Понимание потока документа

Строчный элемент 1

Строчный элемент 2

Блочный элемент (начинает новую строку)

Строчный элемент 3

Строчный элемент 4

```
<h1>Понимание потока документа</h1>
```

```
<p class="inline-example">Строчный элемент 1</p>
```

```
<p class="inline-example">Строчный элемент 2</p>
```

```
<div class="block-example">Блочный элемент (начинает новую строку)</div>
```

```
<p class="inline-example">Строчный элемент 3</p>
```

```
<p class="inline-example">Строчный элемент 4</p>
```

# Понимание потока документа

- это концепция, которая описывает порядок, в котором элементы отображаются на странице.

3

# ВНЕШНИЕ, ВСТРОЕННЫЕ И ИНЛАЙН-СТИЛИ ИМПОРТ



При этом подходе CSS пишется в отдельном файле с расширением .css. Такие стили называют связанными. Для связывания страницы с файлом стилей используется тег <link> внутри <head>

Атрибут rel="stylesheet" указывает, что этот файл является таблицей стилей.

```
<link rel="stylesheet" href="style.css">
```

В href указывается относительный или абсолютный путь к CSS-файлу



- Один CSS-файл для нескольких страниц
- Кэширование браузером
- Использование псевдоклассов и псевдоэлементов



Чтение стилей приостанавливает чтение кода страницы (если файл, к которому обращаетесь, находится на отдалённом и недоступном сервере, это может повлиять на работоспособность)

## Внешний файл со стилями

```
<link href="https://example.com/styles.css" rel="stylesheet">
```

Вариант, когда CSS-свойства описываются в самом HTML-документе внутри тега <style> в шапке страницы



- Браузер не загружает сторонние файлы
- Встроенные стили работают изолированно и применяются непосредственно к странице
- Можно использовать псевдоэлементы



- С каждым новым правилом вес HTML-файла будет увеличиваться и страница будет загружаться медленнее.
- Стили могут понадобиться на других страницах, CSS придётся дублировать.

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="wi
  <title>Document</title>
  <style>
    h1 {
      color: ■tomato;
    }

    p {
      font-size: 24px;
    }
  </style>
</head>
```

## Встроенные стили

В этом варианте элементам, которые необходимо оформить, добавляется атрибут `style`. Значением этого атрибута могут быть свойства и значения CSS, применяемые к элементу.



- Браузеру не нужно запрашивать файл, что ускоряет рендеринг.
- Не нужно писать селекторы (стили применяются сразу к нужному элементу).
- Удобно отлаживать код или делать быстрые правки.
- У инлайн-стилей наивысший приоритет, их можно переопределить только с `!important`.
- Если элементов на странице много, прописывать стили вручную станет неудобно. Такое оформление нарушает принцип «разделения содержимого и оформления».
- Нельзя использовать псевдоклассы и псевдоэлементы.
- Восприятие кода снижается. Код перегружен и менее информативен

## Инлайн-стили

```
<h1 style="color: yellow; margin: 10px 20px">Заголовок</h1>  
<p style="text-align: right; font-size: 12px">Текст</p>
```

Однако несмотря на преимущества, инлайн-стили — это антипаттерн. Недостатков у этого подхода слишком много.

- Даём ссылку на CSS-файл через тег `<link>`.
- Затем в *main.css* импортируем несколько других CSS-файлов, *header.css* для шапки сайта, *navbar.css* для меню, и т.д.

```
<link rel="stylesheet" href="main.css">
```

```
@import url("header.css");  
@import url("navbar.css");  
@import url("footer.css");  
@import url("grid.css");
```



По развития проекта, поддержка CSS-файлов тоже усложняется. `@import` помогает разбить массивную таблицу стилей на более мелкие, понятные части



- Подключение стилей при помощи CSS-файла, в котором директивы `@import` подключают другие файлы, может значительно увеличить время отрисовки страницы
- Чаще лучше добавить несколько элементов `<link>`.

## Импорт CSS

Этот вариант тоже требует подключения внешнего файла стилей. Этот файл содержит не весь CSS-код разом, а стили разбиты на отдельные файлы и подключаются с помощью директивы `@import`

Директива `@font-face` по своей «анатомии» отличается от стандартных CSS-правил, где есть селектор и применяемые к нему правила.

Основное отличие в том, что на месте селектора активируем функцию, адресованную браузеру.

Внутри этого правила должно быть два обязательных свойства:

- **Название шрифта, которое будем использовать в CSS:** свойство `font-family`. Название может отличаться от настоящего названия этого шрифта.
- **Ссылки на файлы, из которых браузер может взять шрифт:** свойство `src`.

```
@font-face {  
  font-family: "Lexend Peta";  
  src: local("Lexend Peta Regular"),  
       local("LexendPeta-Regular"),  
       url("Lexend_Peta-webfont.woff2") format("woff2"),  
       url("Lexend_Peta-webfont.woff") format("woff"),  
       url("Lexend_Peta-webfont.svg") format("svg");  
  font-weight: normal;  
}  
  
body {  
  font-family: "Lexend Peta", sans-serif;  
}
```

## @font-face

Встроенных в операционные системы шрифтов часто не хватает для создания уникального дизайна сайта. Для подключения нестандартных шрифтов существует директива `@font-face`.

```

@font-face {
  font-family: "Lexend Peta Regular";
  src: local("Lexend Peta Regular"),
       local("LexendPeta-Regular"),
       url("Lexend Peta-webfont.woff2") format("woff2"),
       url("Lexend Peta-webfont.woff") format("woff"),
       url("Lexend Peta-webfont.svg") format("svg");
}

@font-face {
  font-family: "Lexend Peta Bold";
  src: local("Lexend Peta Bold"),
       local("LexendPeta-Bold"),
       url("Lexend Peta Bold-webfont.woff2") format("woff2"),
       url("Lexend Peta Bold-webfont.woff") format("woff"),
       url("Lexend Peta Bold-webfont.svg") format("svg");
}

body {
  font-family: "Lexend Peta Regular", sans-serif;
}

.title {
  font-family: "Lexend Peta Bold", sans-serif;
}

```

```

@font-face {
  font-family: "Lexend Peta";
  src: local("Lexend Peta Regular"),
       local("LexendPeta-Regular"),
       url("Lexend Peta-webfont.woff2") format("woff2"),
       url("Lexend Peta-webfont.woff") format("woff"),
       url("Lexend Peta-webfont.svg") format("svg");
  font-weight: normal;
}

@font-face {
  font-family: "Lexend Peta";
  src: local("Lexend Peta Bold"),
       local("LexendPeta-Bold"),
       url("Lexend Peta Bold-webfont.woff2") format("woff2"),
       url("Lexend Peta Bold-webfont.woff") format("woff"),
       url("Lexend Peta Bold-webfont.svg") format("svg");
  font-weight: bold;
}

```

```

body {
  font-family: "Lexend Peta", sans-serif;
  font-weight: normal;
}

.title {
  font-weight: bold;
}

```

## @font-face

Встроенных в операционные системы шрифтов часто не хватает для создания уникального дизайна сайта. Для подключения нестандартных шрифтов существует директива @font-face.

# 4

## CSS СЕЛЕКТОРЫ

### SELECTORS

p

#id

.class

a:link

a:active

a:hover

a:visited

Когда нужно применить одни и те же правила к определённым тегам, используется селектор по тегу. Он применяется ко всем тегам с таким названием вне зависимости от вложенности.



Старайтесь не злоупотреблять селектором по тегу. Правила, написанные для тега, будут применены ко всем таким тегам на странице вне зависимости от вложенности и заданных атрибутов. Это может привести к нежелательным последствиям.

Так, задав стиль подчеркивания для всех ссылок при помощи селектора по тегу, можно получить появление нижнего подчёркивания не только у текстовых ссылок, но и любых других (например, ссылок с иконкой, либо ссылочных картинок)

```
<p>При подготовке к экзамену нужно <span>особое внимание</span> уделить:</p>
<ul>
  <li>основным определениям;</li>
  <li><span>ключевым формулам</span>;</li>
  <li>историческим датам.</li>
</ul>
<p>И обязательно <span>повторить сложные темы</span>.</p>
```

```
span {
  background-color: #E3F2FD;
  border-left: 4px solid #2196F3;
  padding: 2px 8px;
  font-style: italic;
}
```

При подготовке к экзамену нужно *особое внимание* уделить:

- основным определениям;
- *ключевым формулам* ;
- историческим датам.

И обязательно *повторить сложные темы* .

[Пульс РТУ МИРЭА](#)

## Селектор по тегу

Селектор по тегу находит элемент на странице по имени тега



Самое важное – поставить точку перед значением атрибута **class** того элемента, к которому хотите применить стили.

Селектор без точки в начале – **селектор по тегу**.

При подготовке к экзамену нужно **особое внимание** уделить:

- основным определениям;
- **ключевым формулам** ;
- **историческим датам** .

И обязательно **повторить сложные темы** .

```
<p>При подготовке к экзамену нужно <span>особое внимание</span> уделить:</p>
<ul>
  <li>основным определениям;</li>
  <li><span>ключевым формулам</span>;</li>
  <li><span class="tech">историческим датам</span>.</li>
</ul>
<p>И обязательно <span>повторить сложные темы</span>.</p>
```

```
.tech {
  background-color: #F5F5F5;
  border: 1px solid #BDBDBD;
  font-family: "Courier New", monospace;
  padding: 2px 6px;
  border-radius: 2px;
  color: #37474F;
}
```



```
tech {
  background-color: #F5F5F5;
  border: 1px solid #BDBDBD;
  font-family: "Courier New", monospace;
  padding: 2px 6px;
  color: #37474F;
}
```

При подготовке к экзамену нужно **особое внимание** уделить:

- основным определениям;
- **ключевым формулам** ;
- **историческим датам** .

И обязательно **повторить сложные темы** .

Особенности атрибута **class** дают большую гибкость в стилизации. Всегда можно придумать уникальное значение для **class** и по этому значению точно применить нужные стили. Но также можно задать элементу любое количество классов, просто перечислив их через пробел. Благодаря этому можно переиспользовать некоторые стили на разных элементах.

## Селектор по классу

Селектор по классу находит элемент на странице по значению атрибута **class**

В этом примере в HTML есть 2 элемента с разными идентификаторами. В CSS для элемента с идентификатором main-point прописываем, что цвет текста должен быть красным, второму задаём рамку зелёного цвета шириной в 2 пикселя.

```
<p>Важно <span id="main-point">основное</span>  
и <span id="deadline">сроки</span>.</p>
```

Важно **основное** и **сроки**.

```
#main-point {  
  background: #FF6B6B;  
  color: white;  
  padding: 2px 6px;  
}  
  
#deadline {  
  border: 2px solid #4ECDC4;  
  padding: 1px 4px;  
}
```



- Идентификатор чувствителен к регистру. Для браузера `id="myTag"` и `id="MyTag"` будут двумя разными идентификаторами.
- В имени идентификатора нельзя использовать пробел.
- Селектор по идентификатору имеет **очень высокую специфичность**.

```
<ul>  
  <li class="item" id="must-do">Обязательно</li>  
  <li class="item" id="optional">По желанию</li>  
</ul>
```

```
.item {  
  padding: 4px;  
  background: #4ECDC4;  
}
```

- Обязательно
- По желанию

```
#must-do {  
  background: #FFEAA7;  
  border-left: 3px solid #FDCB6E;  
}  
  
#optional {  
  background: #DFE6E9;  
  color: #636E72;  
}
```

## Селектор по идентификатору

Этот селектор находит на странице элемент, которому задан атрибут `id` с конкретным значением

Используются, если нужно задать разные стили для элемента на всей странице и для элемента внутри определённого блока.



Часто пробел не имеет особого значения.

Но не в CSS. При написании стилей пробел

между селекторами является *указателем взаимоотношений* между элементами.

При чтении вложенного селектора первая часть указывает на родительский элемент, а последующие — на вложенные.

Вложенный селектор имеет больший вес, чем селектор, состоящий из одного класса. Поэтому могут возникнуть сложности с переопределением стилей.

```
<article class="product-card">
  <h3>Смартфон</h3>
  <div class="details">
    <p>Характеристики: <span>8GB RAM, 128GB</span></p>
    <div class="price-info">
      <span class="price">25 000 ₽</span>
      <span class="old-price">30 000 ₽</span>
    </div>
  </div>
</article>
```

```
.product-card h3 {
  color: #2D3436;
  margin-bottom: 12px;
}
```

```
.product-card .price-info .price {
  color: #00B894;
  font-size: 1.2em;
  font-weight: bold;
}
```

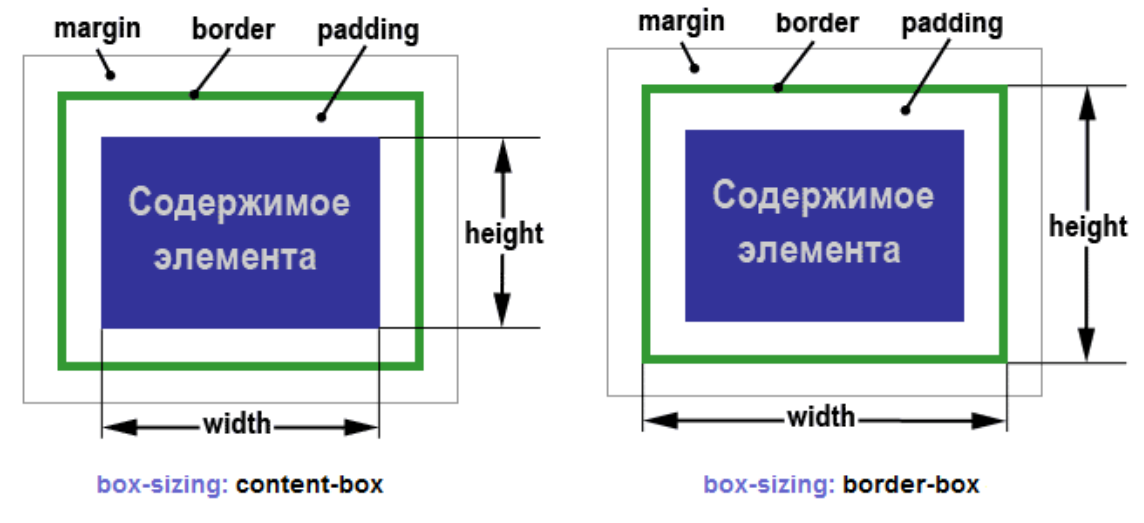
### Смартфон

Характеристики: 8GB RAM, 128GB

**25 000 ₽** 30 000 ₽

## Селектор потомка

Цепочка перечисленных через пробел селекторов обозначает вложенность от родительских элементов к потомку. Это позволяет управлять стилями вложенных элементов.



Универсальный селектор очень удобен, если какие-то свойства нужно применить ко всем элементам на сайте.

Некоторые используют универсальный селектор для изменения алгоритма расчёта размеров элементов перед началом вёрстки

Учитывайте, что это CSS-правило не будет иметь влияния на псевдоэлементы. Алгоритм расчёта нужно будет менять точно. Или дополнить селектор, упомянув в нём `::before` и `::after`

```
*,
::before,
::after {
  box-sizing: border-box;
}
```

## Универсальный селектор

Универсальный селектор `*` соответствует абсолютно любому тегу, но не включает псевдоэлементы

<https://habr.com/ru/companies/otus/articles/580442/>

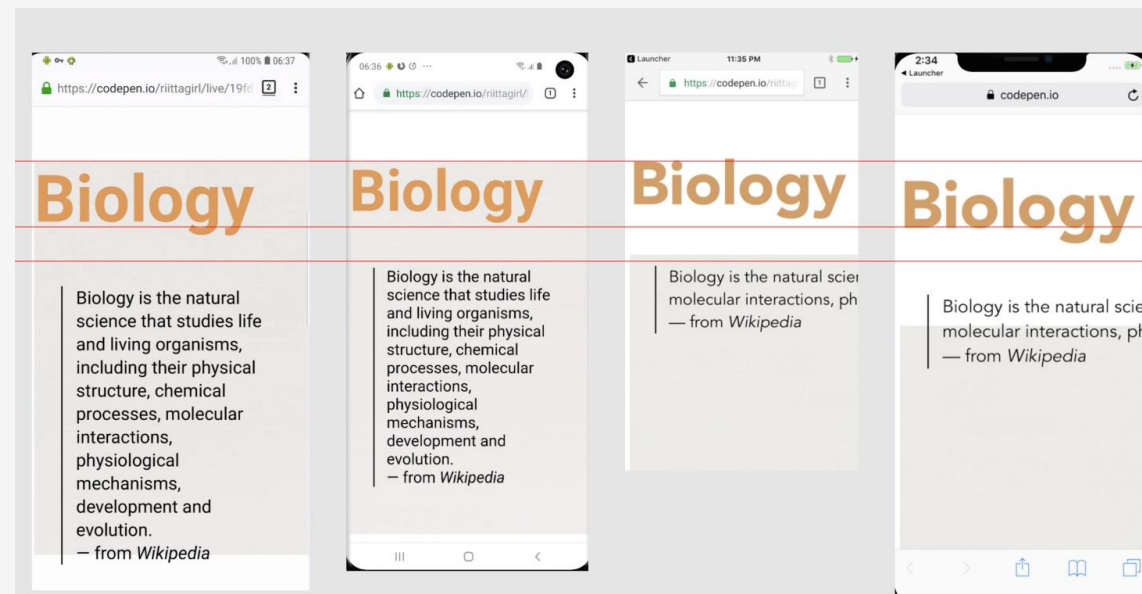
# Универсальный селектор и сброс стилей

Чем тщательнее вы подготовитесь (dev-ops, пайплайны, бандлинг заказов) к проекту заранее, тем больше времени сэкономите в долгосрочной перспективе, то есть наличие правильных начальных настроек для reset.css потенциально может сэкономить вам часы отладки вашего CSS.

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

По мере формирования страницы вы будете добавлять стили CSS. Создав раздел, проверьте его в разных браузерах, между ними вы заметите несколько небольших расхождений. Эти мелкие несоответствия накапливаются со временем и в итоге дизайн вашего сайта различен во всех браузерах.

**Именно по этой причине люди используют сброс настроек/стилей: для единообразия.**



Универсальный селектор можно использовать в комбинированных селекторах, ограничивая его область действия.

Такое CSS-правило применит цвет текста ко всем элементам, вложенным в блок с классом **.product-info:**

Изменим селектор на вложенный, и цвет изменится у всех прямых потомков блока с классом **.product-info:**

Учитывайте, что это CSS-правило не будет иметь влияния на псевдоэлементы.

Алгоритм расчёта нужно будет менять точно. Или дополнить селектор, упомянув в нём `::before` и `::after`

```
<article class="product-info">
  Название товара
  <div class="price">5000 Р</div>
  <div class="features">
    <span>Характеристика 1</span>
    <span>Характеристика 2</span>
  </div>
</article>
```

```
.product-info {
  color: green;
}
```

Название товара  
5000 Р  
Характеристика 1 Характеристика 2

```
.product-info > * {
  color: green;
}
```

Название товара  
5000 Р  
Характеристика 1 Характеристика 2

## Комбинированные селекторы

Можно использовать \* в смежном селекторе, чтобы правило применялось только в случае, если рядом с элементом есть сосед. Например, такое CSS-правило задаст нижний отступ всем элементам при условии, что они не являются единственными детьми своего родителя

```
* + * {  
  margin-bottom: 15em;  
}
```



Не стоит злоупотреблять универсальным селектором.

Есть вероятность столкнуться с необходимостью постоянно переопределять заданные с его помощью свойства.



## Универсальный селектор

«Лоботомированная сова»

Этот тип селектора помогает нам стилизовать элементы, опираясь либо на наличие самого атрибута, либо на его значение.

После элемента в квадратных скобках указывается атрибут. При выборке по значению дополнительно указывается, что атрибут должен быть равен значению в кавычках.



```
<blockquote cite="А. С. Пушкин">
  Октябрь уж наступил — уж роща отряхает<br>
  Последние листья с нагих своих ветвей;
</blockquote>

<blockquote>
  Октябрь уж наступил — уж роща отряхает<br>
  Последние листья с нагих своих ветвей;
</blockquote>
```

```
blockquote[cite] {
  background-color: #2E9AFF;
}
```

Октябрь уж наступил — уж роща отряхает  
Последние листья с нагих своих ветвей;

Октябрь уж наступил — уж роща отряхает  
Последние листья с нагих своих ветвей;

## Селектор по атрибуту

Селектор по атрибуту находит элемент на странице по значению либо по наличию атрибута.



## Группировка

Если нескольким селекторам нужно задать одно и то же правило, то можно написать длинно

1

```
h1, h2, h3, #card {  
  font-weight: bold;  
}
```

## Объединение

Только для классов и атрибутов, потому что только их может быть больше одного. Селекторы записываются слитно. Стили будут применены только к тому элементу, который содержит все перечисленные селекторы.

2

```
.selector.card {  
  font-weight: bold;  
}
```

3

```
article h3 {  
  font-weight: bold;  
}
```

## Потомки

Последовательность селекторов отражает вложенность — каждый следующий должен находиться на каком-то уровне вложенности в предыдущий селектор

# Комбинированные селекторы

Иногда при написании стилей мы хотим обратиться к селектору более точно, чем просто по имени класса или тега. Для этого можно использовать различные комбинации.

## Непосредственно вложенные

Последовательность селекторов отражает непосредственную вложенность — селектор справа от оператора должен быть прямым потомком селектора из левой части

## Смежные

Элемент справа от + должен следовать в HTML сразу за элементом слева от +. Проще говоря, правый элемент должен быть соседом левого элемента, чтобы смежный селектор сработал.

4

```
article > h3 {  
  color: red;  
}
```

5

```
label + input {  
  color: red;  
}
```

6

```
.star:hover ~ .star {  
  color: red;  
}
```

## Последующие

Применится ко всем блокам, подходящим под правый селектор, при учёте, что они являются соседями блоков из селектора слева от ~. Оба селектора должны иметь одного родителя, находиться на одном уровне вложенности. В HTML правый селектор должен идти после левого.

# Комбинированные селекторы

Иногда при написании стилей мы хотим обратиться к селектору более точно, чем просто по имени класса или тега. Для этого можно использовать различные комбинации.

```
<header class="site-header">
  <h1>Заголовок сайта</h1>
  <nav id="main-nav">Навигация</nav>
</header>

<main class="content">
  <article class="post">
    <h2>Статья</h2>
    <p>Текст статьи</p>
  </article>
</main>

<footer class="site-footer">
  <p>Копирайт</p>
</footer>

.site-header,
#main-nav,
.content h2,
footer.site-footer {
  background-color: #f8f9fa;
  padding: 20px;
  margin-bottom: 15px;
  border-radius: 8px;
  border-left: 4px solid #2E9AFF;
}
```

Указывается один селектор, после него ставится запятая, через пробел указывается второй селектор и так далее. Для удобства можно переносить каждый новый селектор на новую строку, технически это не обязательно.

Селекторами может быть не только селектор по классу, но и селектор по тегу или по идентификатору, или даже их комбинация.



Если стили у перечисляемых блоков не отличаются ничем, то это причина пересмотреть разметку и задать одинаковым по стилям элементам одинаковый класс.

Если перечисляете через запятую составные селекторы, то нужно указывать их полностью.

## Перечисление селекторов

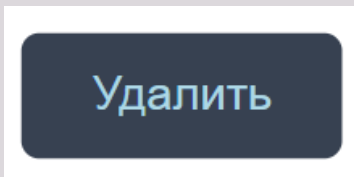
При помощи перечисления нескольких селекторов через запятую можно избежать дублирования кода

# ПСЕВДОКЛАССЫ

**Псевдоклассы** — особый вид селектора, который уточняет тип или состояние. Обычно это какой-то качественный признак: реакция на наведение курсора, порядок следования и другие.

Благодаря псевдоклассам мы можем контролировать динамические параметры селекторов. Эти свойства сработают, когда селектор подходит под дополнительный признак.

```
:focus {  
  color: lightblue;  
}
```



```
<div class="actions">  
  <button class="btn">Добавить в корзину</button>  
  <button class="btn btn-primary">Купить сейчас</button>  
  <button class="btn btn-danger">Удалить</button>  
</div>
```

Добавить в корзину

Купить сейчас

Удалить

```
.btn:hover {  
  background-color: #4B5563;  
  transform: scale(1.05);  
}  
  
.btn-primary:hover {  
  background-color: #1E40AF;  
}
```

Купить сейчас

Селектор может и отсутствовать. Тогда правило применится ко всем элементам, которые могут иметь признак этого псевдокласса.

Так, CSS-правило **:focus {}** применится к любому элементу, который будет в фокусе.

# ПСЕВДОКЛАССЫ

## :active

Применяется к интерактивным элементам в момент взаимодействия.

Например, нажатая кнопка

## :is()

Позволяет сгруппировать схожие селекторы вместо перечисления через запятую. При группировке селекторов может сократить и упростить написание

```
h1 a, h2 a, h3 a, h4 a, h5 a, h6 a {  
  color: lightblue;  
}  
  
:is(h1, h2, h3, h4, h5, h6) a {  
  color: lightblue;  
}
```



## :any-link, :link, :visited

Применяется ко всем элементам, которые могут иметь атрибут href (<a>, <area> и <link>).

:link характеризует не посещённые страницы, :visited – посещённые

## :checked

Применяется к элементам, состояние которых меняется с помощью checked



UNCHECKED



HOVER



CHECKED

# ПСЕВДОКЛАССЫ

## :dir()

Находит элементы по направлению текста в нём. С 2024 года поддерживается во всех основных браузерах.

***h1:dir(ltr) { background: red; }*** — для текста, идущего слева направо

## :default

Применяется к элементам формы (<input type="radio">, <input type="checkbox">, <option> и <button>), у которых можно задать начальное состояние.

## :disabled, :enabled

Позволяют находить элементы формы по состоянию их атрибута disabled.

Реагируют на унаследованное состояние disabled. Если есть <fieldset disabled>, то отключаются вложенные в него контроли форм.

## Пиццерия «Нам видней»

Выбрать можно только сыр.

<fieldset disabled>

Томатный соус

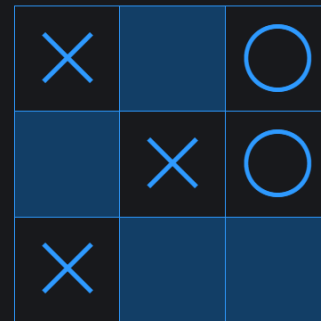
<input disabled>

Ветчина, ананас, маслины

:enabled

Двойной сыр

# ПСЕВДОКЛАССЫ



## :empty

Применяется к элементам, у которых внутри нет других тегов или текста. Так, можно проверить, что у кнопки не задан текст или иконка, чтобы задать минимальные размеры

## :fullscreen

Признак того, что документ развернули на всё окно (с помощью JavaScript).

Скроем панель управления у плеера, если он развёрнут на весь экран

```
.player:fullscreen .player__controls {  
  display: none;  
}
```

## n<sup>th</sup> Child Selectors

in CSS

|                   |                                     |                                     |                                     |                                     |                                     |                                     |
|-------------------|-------------------------------------|-------------------------------------|-------------------------------------|-------------------------------------|-------------------------------------|-------------------------------------|
| first-child       | <input type="checkbox"/>            | <input type="checkbox"/>            | <input type="checkbox"/>            | <input type="checkbox"/>            | <input type="checkbox"/>            | <input type="checkbox"/>            |
| last-child        | <input type="checkbox"/>            | <input type="checkbox"/>            | <input type="checkbox"/>            | <input type="checkbox"/>            | <input type="checkbox"/>            | <input checked="" type="checkbox"/> |
| nth-child(2)      | <input type="checkbox"/>            | <input checked="" type="checkbox"/> | <input type="checkbox"/>            | <input type="checkbox"/>            | <input type="checkbox"/>            | <input type="checkbox"/>            |
| nth-last-child(2) | <input type="checkbox"/>            | <input type="checkbox"/>            | <input type="checkbox"/>            | <input type="checkbox"/>            | <input checked="" type="checkbox"/> | <input type="checkbox"/>            |
| nth-child(odd)    | <input checked="" type="checkbox"/> | <input type="checkbox"/>            | <input checked="" type="checkbox"/> | <input type="checkbox"/>            | <input checked="" type="checkbox"/> | <input type="checkbox"/>            |
| nth-child(even)   | <input type="checkbox"/>            | <input checked="" type="checkbox"/> | <input type="checkbox"/>            | <input checked="" type="checkbox"/> | <input type="checkbox"/>            | <input checked="" type="checkbox"/> |
| nth-child(n+4)    | <input type="checkbox"/>            | <input type="checkbox"/>            | <input type="checkbox"/>            | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |
| nth-child(3n-1)   | <input type="checkbox"/>            | <input checked="" type="checkbox"/> | <input type="checkbox"/>            | <input type="checkbox"/>            | <input checked="" type="checkbox"/> | <input type="checkbox"/>            |
| nth-child(3n+1)   | <input checked="" type="checkbox"/> | <input type="checkbox"/>            | <input type="checkbox"/>            | <input checked="" type="checkbox"/> | <input type="checkbox"/>            | <input type="checkbox"/>            |
| nth-child(4n)     | <input type="checkbox"/>            | <input type="checkbox"/>            | <input type="checkbox"/>            | <input checked="" type="checkbox"/> | <input type="checkbox"/>            | <input type="checkbox"/>            |

## Псевдоклассы группы child

Селекторы элемента по его положению относительно родителя (первый, второй, последний, единственный, n-й и т.д.)

## Псевдоклассы группы type

Селектор по типу внутри одного родителя

**:nth-of-type()**

# ПСЕВДОКЛАССЫ

## :focus и :focus-within

Элемент, который сейчас в фокусе.

А **:focus-within** ещё обозначает элемент, внутри которого находится элемент в фокусе.



Warning only visible when you're focusing some element!

## :has()

Позволяет выбрать элемент, уточнив основной селектор дополнительным, не увеличив его вес. Правило применится только к тому селектору, который был описан до :has()

***div:has(img) { border: 2px solid blue; }*** - задаёт стили границ только для тех элементов <div>, которые содержат внутри себя изображения.

## :hover

Состояние элемента, когда на него навели курсор





# ПСЕВДОКЛАССЫ

## :in-range и :out-of-range

Применяется для `<input>`, у которого определены атрибуты `min` и `max` и введенное значение соответствует (`:in-range`) или нет (`:out-of-range`) этому диапазону



## :lang()

Селектор по языку содержимого. В HTML `lang`, указывает на язык содержимого. Псевдокласс `lang()` позволяет обратиться к элементу, чье значение атрибута `lang` подходит под условие.

```
:lang(en) {  
  border-left-color: #3B82F6;  
  background-color: #FFF6FF;  
  quotes: "«" "»" "„" "”";  
}
```

```
:lang(ru) {  
  border-left-color: #8B5CF6;  
  background-color: #F5F3FF;  
  quotes: "«" "»";  
}
```

Возьми всё:

☐ Турка

☐ Вода

☐ Кофе

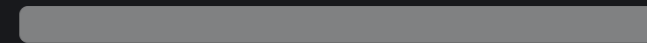
Выбери одно:

☐ Корица

☐ Ваниль

☐ Лаванда

Настроение:



Выпить кофе

Возьми всё:

☒ Турка

☐ Вода

☐ Кофе

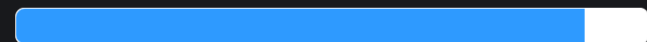
Выбери одно:

☐ Корица

☐ Ваниль

☒ Лаванда

Настроение:



Удалить значение

## :indeterminate

Элементы, которые могут находиться в промежуточном состоянии:

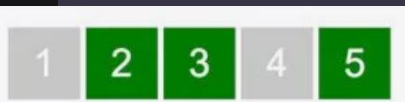
- `<input type="checkbox">`, когда не все пункты вложенной группы были выделены.
- Группа `<input type="radio">` с одинаковым `name`, но ни один элемент не был выбран.
- `<progress>`

# ПСЕВДОКЛАССЫ

## :not()

Находит элемент, который не соответствует селектору, переданному в качестве аргумента.

```
p:not(:first-of-type):not(.hola) {  
  background-color: green;  
}
```



## :root

Соответствует корневому тегу-элементу документа. Для HTML это, соответственно, <html>, для SVG — <svg>.

Специфичность селектора :root выше, чем у селектора по тегу

```
/* Переопределит цвет */  
:root {  
  --color: blue;  
}  
  
/* :root = <html> */  
html {  
  --color: red;  
}
```

## :optional и :required

:optional находит любой <input>, у которого не установлен атрибут required (находит необязательные поля ввода).

А :required те <input>, у которых есть этот атрибут

Marking required fields makes users skip optional fields



✗ Bad

Marking optional fields won't inhibit user's voluntary over-disclosure



✓ Good

# ПСЕВДОКЛАССЫ

## :read-only и :read-write

Находит элементы, недоступные (:read-only) / доступные (:read-write) для редактирования. Например, ориентируется на наличие атрибута disabled или readonly.

## :read-write Selector

Editable input feild

This is a read-only input field.

## :valid и :invalid

Селектор :valid соответствует <input>, <fieldset> или <form>-элементу, контент которого валиден в соответствии с типом поля или полей внутри.

Обратный эффект у :invalid — сработает при ошибке HTML-валидации

```
input:valid {  
  border-color: #28a745;  
  background: #f8fff8;  
}  
  
input:invalid {  
  border-color: #dc3545;  
  background: #fff5f5;  
}
```

### Live Form Demo

Email Address

user@example.com



Phone Number

invalid-phone



Website URL

https://example.com



VALID



INVALID

# ПСЕВДОКЛАССЫ

## :target

При переходе по ссылке, которая ведёт на URI-фрагмент (элемент внутри страницы), id фрагмента совпадает со значением атрибута id этого элемента — это состояние можно «поймать» с помощью псевдокласса **:target**

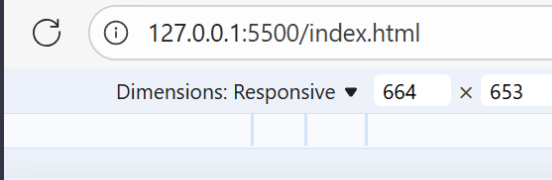
```
<a href="#section1">Раздел 1</a>
<a href="#section2">Раздел 2</a>

<article id="section1" class="section">
  <h2>Раздел 1</h2>
  <p>Содержание раздела 1</p>
</article>

<article id="section2" class="section">
  <h2>Раздел 2</h2>
  <p>Содержание раздела 2</p>
</article>
```

```
.section {
  padding: 20px;
  margin: 10px;
  background: #f0f0f0;
  transition: all 0.3s ease;
}

/* Стил ь применяется к элементу,
на который ведёт якорная ссылка */
.section:target {
  background: #2E9AFF;
  color: white;
  border: 3px solid #1A6FCC;
  transform: scale(1.02);
}
```



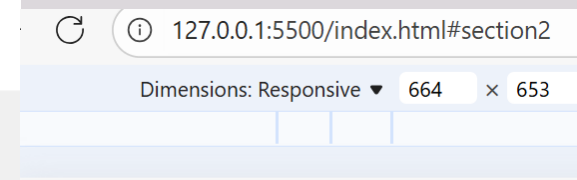
[Раздел 1](#) [Раздел 2](#)

## Раздел 1

Содержание раздела 1

## Раздел 2

Содержание раздела 2



[Раздел 1](#) [Раздел 2](#)

## Раздел 1

Содержание раздела 1

## Раздел 2

Содержание раздела 2

5

# СПЕЦИФИЧНОСТЬ

When your **CSS** property doesn't work



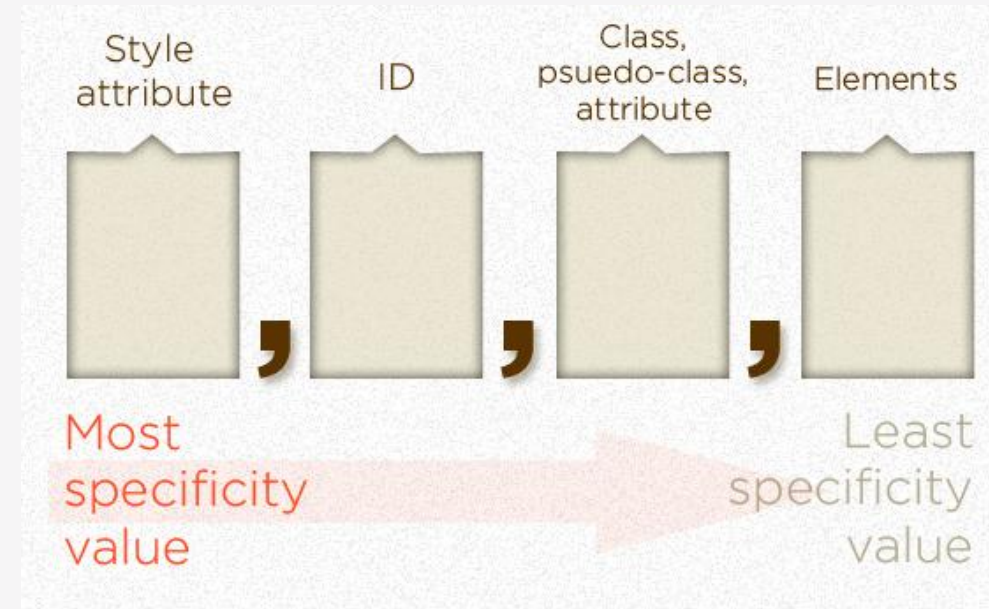
**!important**



**Специфичность** — это алгоритм, благодаря которому браузер определяет, какие именно стили из всего набора применить к элементу. В вычислениях участвуют CSS-селекторы.

Если одному и тому же элементу подходит сразу несколько CSS-правил с разными селекторами, то браузер применяет те стили, вес селектора которых больше.

Правило каскада «кто ниже, тот и выигрывает» при этом может нарушаться



## Специфичность

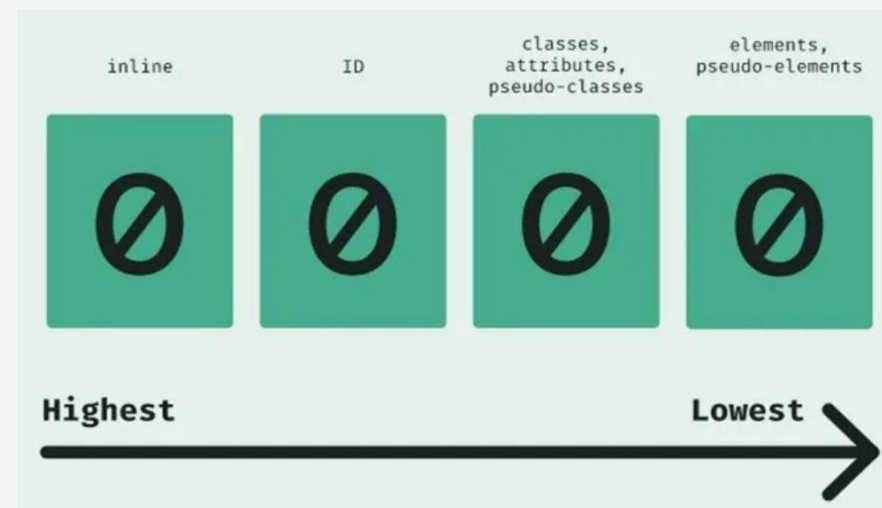
Специфичность — это одно из базовых понятий в CSS

## Типы селекторов по убыванию специфичности:

1. Селекторы по идентификатору.
2. Селекторы по классу, селекторы по атрибуту и селекторы с псевдоклассами.
3. Селекторы по тегу, селекторы с псевдоэлементами.

Комбинаторы `+`, `>`, `~`, универсальный селектор `*` и псевдокласс `:where()` веса не имеют.

Псевдоклассы `:is()`, `:has()` и `:not()` принимают вес наиболее специфичного селектора внутри скобок.



## Вес селекторов

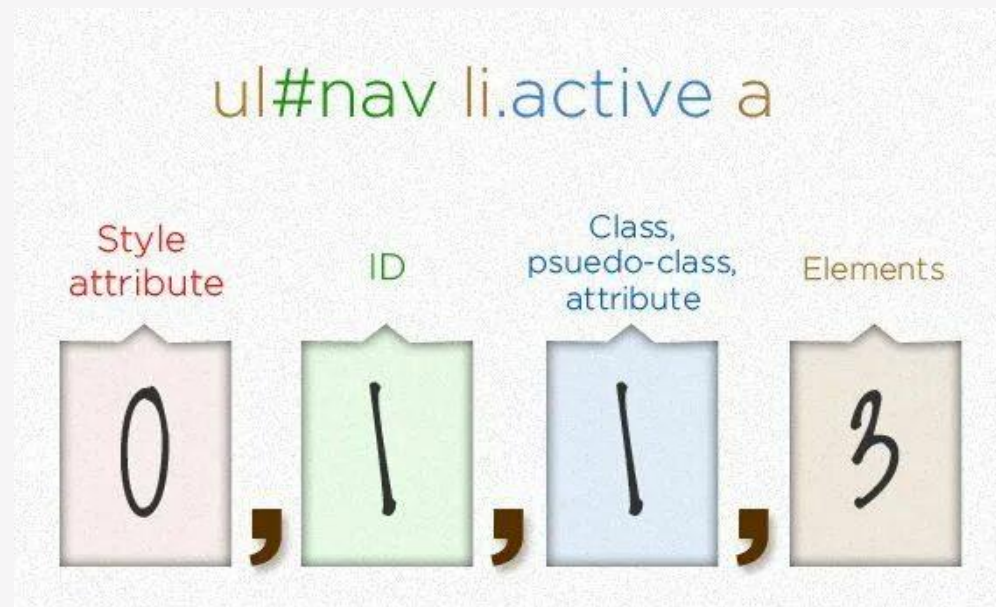
Специфичность — это одно из базовых понятий в CSS



## Система расчёта

Ранее перечислили три группы сущностей, из которых может состоять селектор. Представим любой селектор в виде трёх нулей: 0.0.0.

- Селекторы по идентификатору увеличивают **первую** цифру.
- Селекторы по классу, по атрибуту или псевдокласс увеличивают **вторую** цифру.
- Селектор по тегу или псевдоэлемент увеличивают **третью** цифру.





Универсальный селектор \* ничего не весит

## Система расчёта на примерах

|              |            |
|--------------|------------|
| style=""     | 1, 0, 0, 0 |
| #id          | 0, 1, 0, 0 |
| .class       | 0, 0, 1, 0 |
| [attr=value] | 0, 0, 1, 0 |
| LI           | 0, 0, 0, 1 |
| *            | 0, 0, 0, 0 |

### Пусть

- Селектор **#some** состоит из 1 идентификатора.  
Один селектор = 1.  
Увеличиваем первую цифру на 1.  
В итоге вес такого селектора равен 1.0.0.
- .class** состоит из одного класса.  
Увеличиваем вторую цифру на 1.  
Получаем вес селектора 0.1.0.
- section** состоит из одного тега.  
Увеличиваем последнюю цифру на 1.  
Вес селектора равен 0.0.1.

### Комбинируем

- div#some** состоит из селектора по тегу и идентификатора.  
Селектор по тегу увеличивает последнюю цифру,  
селектор по идентификатору — первую.  
Вес селектора равен 1.0.1.
- section h1** состоит из двух селекторов по тегу.  
Увеличиваем последнюю цифру на два. Вес 0.0.2.
- #block section > .list a** состоит из идентификатора (первая цифра), двух тегов (последняя цифра) и класса (вторая цифра). Вес селектора равен 1.1.2.
- \* .list a** состоит из одного класса и одного тега. Вес 0.1.1.

Иногда его добавляют в формулу в виде четвёртой цифры, стоящей перед всеми

# Атрибут style

```
<div class="element" id="this" style="color: purple; border: none">
  Мини текст
</div>
```

```
div.element#this {
  color: green;
  border: 10px solid red;
}
```

Мини текст

У селектора в CSS будет специфичность 1.1.1, потому что там указан один идентификатор, один класс и один селектор тега. Но в итоге текст в блоке будет пурпурным, а рамки не будет совсем. Потому что у атрибута style вес равен 1.0.0.0

|              |            |
|--------------|------------|
| style=""     | 1, 0, 0, 0 |
| #id          | 0, 1, 0, 0 |
| .class       | 0, 0, 1, 0 |
| [attr=value] | 0, 0, 1, 0 |
| LI           | 0, 0, 0, 1 |
| *            | 0, 0, 0, 0 |

Ключевое слово **!important** нарушает все

# !important

**!important** нарушает все установленные спецификацией законы и насильно применяет свойство, после которого написано. Ему плевать на селектор CSS-правила, внутри которого это свойство написано. Будет так, и никак иначе.

Формально это ключевое слово не имеет отношения к концепции специфичности. Но из-за его варварских замашек нельзя не принимать его во внимание.

**!important** всегда стоит использовать с осторожностью и не злоупотреблять им. В том числе из-за того, что оно нарушает естественную работу специфичности и каскада.



6

# КАСКАДНОСТЬ

## Origin & Importance

!browser  
!user  
!author  
author  
user  
browser

## (Shadow) Context

!shadow  
!host  
host  
shadow

## Element-Attached

inline styles

## Cascade Layers

<!first>  
<!last>  
!un-layered  
un-layered  
<last>  
<first>

## Selector Specificity

#ID  
::pseudo | element  
:pseudo | .class | [attr]  
:where() | \*

## Order of Appearance

last style wins

Один из основных принципов работы CSS. Стили читаются и применяются сверху вниз

## Принцип каскада



Само название CSS — **Cascading Style Sheets**, «каскадные таблицы стилей» явно упоминают это неочевидное определение.

Применение каскадности позволяет управлять тем, какие стили будут применены, когда к одному и тому же элементу применяются несколько правил. Основные факторы, влияющие на каскадность, включают **специфичность** селекторов, **порядок определения** стилей и **важность** (использование !important).

## Чем выше – тем «важнее»

- 
1. Стили во время CSS-переходов (transition).
  2. Браузерные стили с !important.
  3. Пользовательские стили с !important.
  4. Авторские (мы с вами пишем именно эти стили) стили с !important.
  5. Стили во время анимаций (animation).
  6. Обычные (т. е. без !important) авторские стили.
  7. Обычные пользовательские стили.
  8. Обычные браузерные стили.

Иногда браузеры нарушают этот порядок — порой осознанно (могут запретить делать слишком мелкий шрифт в полях ввода, как это делал Google Chrome) или по ошибке (до недавнего времени стили при анимации перекрывали остальные стили только в Firefox).

# Область видимости

Понимание лучше начать с указания стилей в атрибуте тега `style` — их область видимости ограничена только тем тегом, в котором они указаны. Т. е. если вы описали `style` тегу `<li>`, то эти стили применятся только к этому тегу, остальные `<li>` не «увидят» эти свойства

- Этот пункт имеет CSS-свойства по умолчанию
- И этот тоже
- Текст в этом (и только в этом) теге будет синим
- Снова обычное оформление

```
<ul>  
  <li>Этот пункт имеет CSS-свойства по умолчанию</li>  
  <li>И этот тоже</li>  
  <li style="color: #2E9AFF">Текст в этом (и только в этом) теге будет синим</li>  
  <li>Снова обычное оформление</li>  
</ul>
```

Со свойствами, описанными через классические CSS-селекторы (классы, теги, `id`, атрибуты) иначе. Чем точнее описываем правило для селектора, тем сильнее мы суживаем область видимости, тем самым увеличивая и приоритетность правила.

Но это работает до тех пор, пока CSS-правило не содержит «подлого» `!important`. Именно поэтому их использование считается плохой практикой — они «меняют правила игры». Это правило мгновенно становится приоритетнее, но чем шире область видимости — тем оно важнее

## Самое простое и очевидное в каскаде

# Специфичность

Когда описываем стили, каждое правило имеет свой вес. Правила, описанные инлайном перебивают правила на селектор, но не отменяют правила с !important (отменяют, если инлайн-правило тоже описано с !important)

<p>  
|  
</p>

Цвет текста будет красным – единственное правило описано в теге `<style>`.



```
p {
  color: red;
}
```

10

Цвет текста будет красным — единственное правило описано в теге `<style>`.



```
<p style="color: blue">
    Цвет текста стал синим — инлайн «перебивает» <style>.
</p>
```

Цвет текста стал синим — инлайн «перебивает» `<style>`.

<p>  
|  
</p>

```
style="color: blue">
Цвет снова красный за счёт !important в <style>.
>
```



```
p {  
  color: red !important;  
}
```

2

Цвет снова красный за счёт  
!important в <style>.



```
<p style="color:  blue !important">
    Цвет снова синий – !important инлайн-правила «перебивает»
    даже !important в &lt;style>;
</p>
```



Цвет снова синий — !important инлайн-правила «перебивают» даже !important в <style>.



## Порядок в коде

При равной специфичности правила, написанные ниже по ходу чтения переопределяют написанное выше

```
<style>
  .color_red {
    /* Определяем начальное значение */
    color: ■red;
  }

  .color_red {
    /* Переопределяем написанное ранее правило */
    color: ■blue;
  }
</style>

<p class="color_red">Цвет текста — синий!</p>
```

Цвет текста — синий!

## Влияние на каскад

### Начальное значение — initial

Любое CSS-правило можно сбросить на начальное значение, которое было у правила до того как было установлено явно разработчиком.

color станет #000, position — static, display — inline, padding и margin — 0 и так далее

Установка initial выставит именно дефолтное значение без учёта значения для тега

# Влияние на каскад

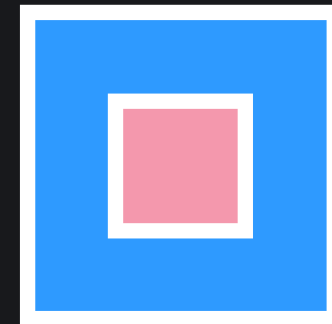
## Заимствование у родителя – `inherit`

Некоторые свойства наследуются от родителя автоматически (`color`, `font`, `text-align` и др.).

Но можно и явно позаимствовать какое-то значение у родителя.

```
.wrapper {  
  background-color: #2E9AFF;  
  border: 10px solid #FFFFFF;  
}
```

```
.inner {  
  background-color: #F498AD;  
  border: inherit;  
}
```



## Отмена значения — `unset`

По своему поведению `unset` чем-то похож на `initial + inherit` – если это свойство ненаследуемое от родителя, то оно сбросится до начального значения, а наследуемое получит значение родителя.

## Сброс значения — `revert`

Это менее строгий режим сброса.

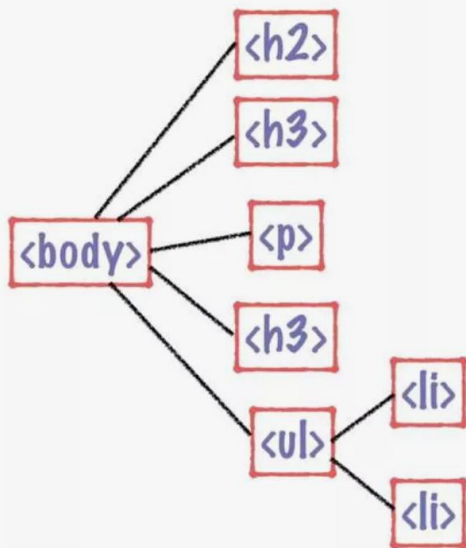
Он отменяет все установленные разработчиком значения для данного свойства до значения, которое считается браузером «значением по умолчанию»

7

# НАСЛЕДОВАНИЕ

Если у какого-то свойства указать значение `inherit` – оно будет взято у верхнего «родителя». Для некоторых CSS-свойств это значение указано по умолчанию

## Наследование в CSS



Наследование в CSS — это механизм, который позволяет элементам наследовать стили от своих родительских элементов.

Это означает, что некоторые свойства CSS, заданные для родительского элемента, автоматически применяются к дочерним элементам, если они не имеют собственных значений для этих свойств

Для некоторых HTML-тегов определены особые CSS-свойства по умолчанию (user agent-стили, или стили от браузера), которые характерны только для конкретно этих элементов, например, **color: #00c** для ссылок.

Их можно переопределить, но некоторые свойства (цвета текста, размера шрифта, внешнего вида курсора и другие) будут применяться для любых вложенных элементов, пока для них не будет указано иного значения. Такая наследуемость не случайна и обусловлена значением у этих свойств – `inherit`,

# Наследуемые свойства

**Свойства шрифта:** font-style, font-variant, font-weight, font-stretch, font-size и font-family; color и line-height.

**Свойства межбуквенных и «межсловных» расстояний:** letter-spacing, word-spacing и white-space.

**Параметры текста:** text-align, text-indent, text-shadow, text-transform;

**Оформление пунктов списков:** list-style, list-style-type, list-style-position.

**Внешний вид курсора:** cursor и отображение содержимого элемента visibility

# Ненаследуемые свойства

Некоторые свойства, такие как width, height, margin, и border, не наследуются

```
<div class="parent">  
  Родительский элемент  
  <div class="child">Дочерний элемент</div>  
</div>
```

```
.parent {  
  width: 300px; /* Ширина родителя */  
  height: 200px; /* Высота родителя */  
  border: 2px solid black; /* Граница родителя */  
  padding: 10px; /* Отступы для родителя */  
}  
.child {  
  background-color: yellow; /* Цвет фона для дочернего
```

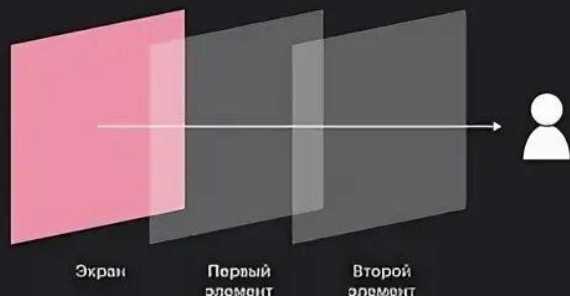
Родительский элемент  
Дочерний элемент

8

## КОНТЕКСТ НАЛОЖЕНИЯ

Контекст наложения – набор правил, на основании которых браузер решает, какой элемент на странице будет выше или ниже других.

## Контекст наложения



Браузер при отрисовке страницы идёт сверху вниз по HTML-разметке и рисует блоки один за другим в том же порядке, в котором они расположены.

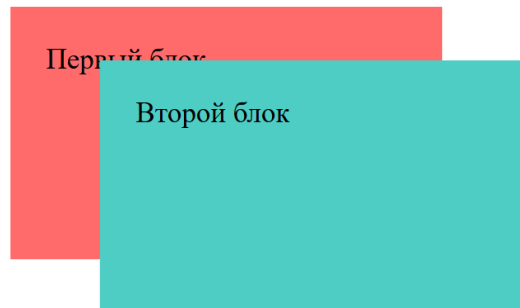
Мы воспринимаем это как плоский лист, где выстраиваются элементы. Но браузер располагает элементы не только по осям x и y, но ещё и по z.

```
.block {  
  width: 200px;  
  height: 100px;  
  padding: 20px;  
  position: absolute;  
}
```

```
.first {  
  background: #FF6B6B;  
  top: 50px;  
  left: 50px;  
}
```

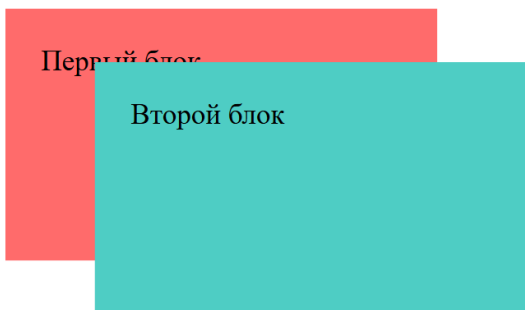
```
.second {  
  background: #4ECDC4;  
  top: 80px;  
  left: 100px;  
}
```

```
<div class="block first">Первый блок</div>  
<div class="block second">Второй блок</div>
```



В результате явно видно, что второй блок перекрывает собою первый, а значит находится выше в стопке элементов по оси z

# z-index



Но мы можем не подчиняться стандартным правилам браузера и устанавливать свой порядок наложения элементов друг на друга.

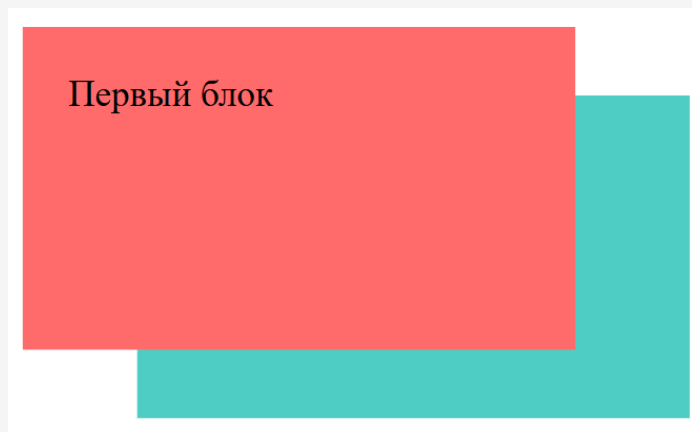
Для этого понадобится свойство `z-index`

Допустим, хотим переместить первый блок на передний план.

Для этого достаточно добавить позиционирование и `z-index` для первого.

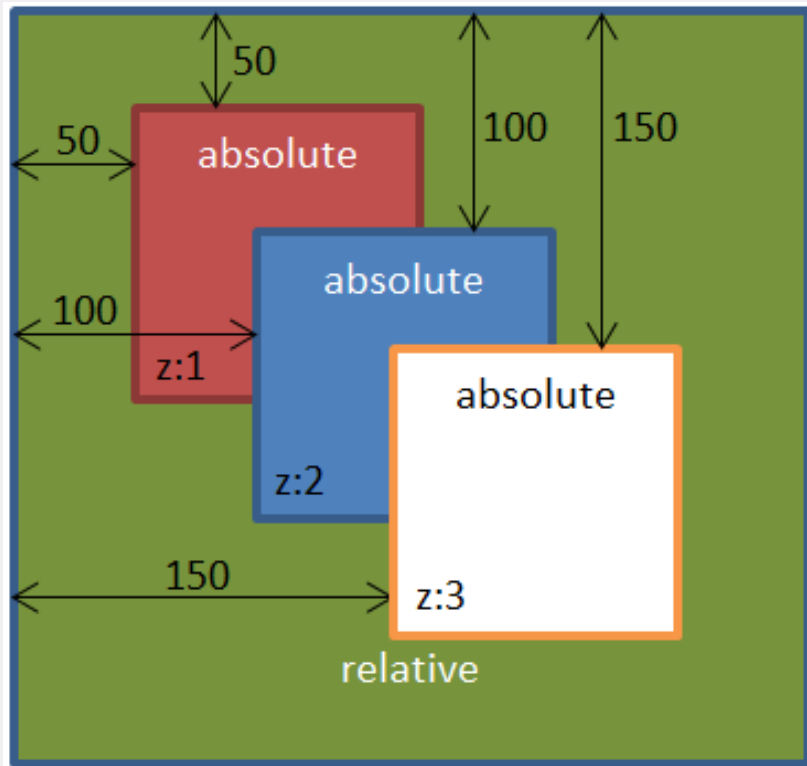
```
.block {  
  width: 200px;  
  height: 100px;  
  padding: 20px;  
  position: absolute;  
}
```

```
.first {  
  background: #FF6B6B;  
  top: 50px;  
  left: 50px;  
  z-index: 1;  
}  
  
.second {  
  background: #4ECDC4;  
  top: 80px;  
  left: 100px;  
}
```





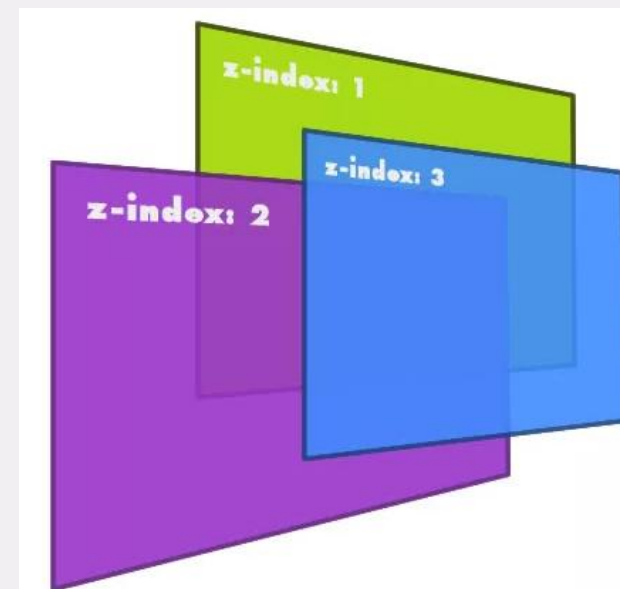
# Контекст наложения



Самый базовый контекст наложения формируется корневым элементом `<html>`.

Все элементы внутри этого контекста сортируются и располагаются с оглядкой друг на друга.

Также можем создавать контексты в каждом отдельном блоке. Тогда дочерние блоки будут сортироваться и располагаться по правилам родительского контекста наложения.



# Новый контекст наложения формируется если:

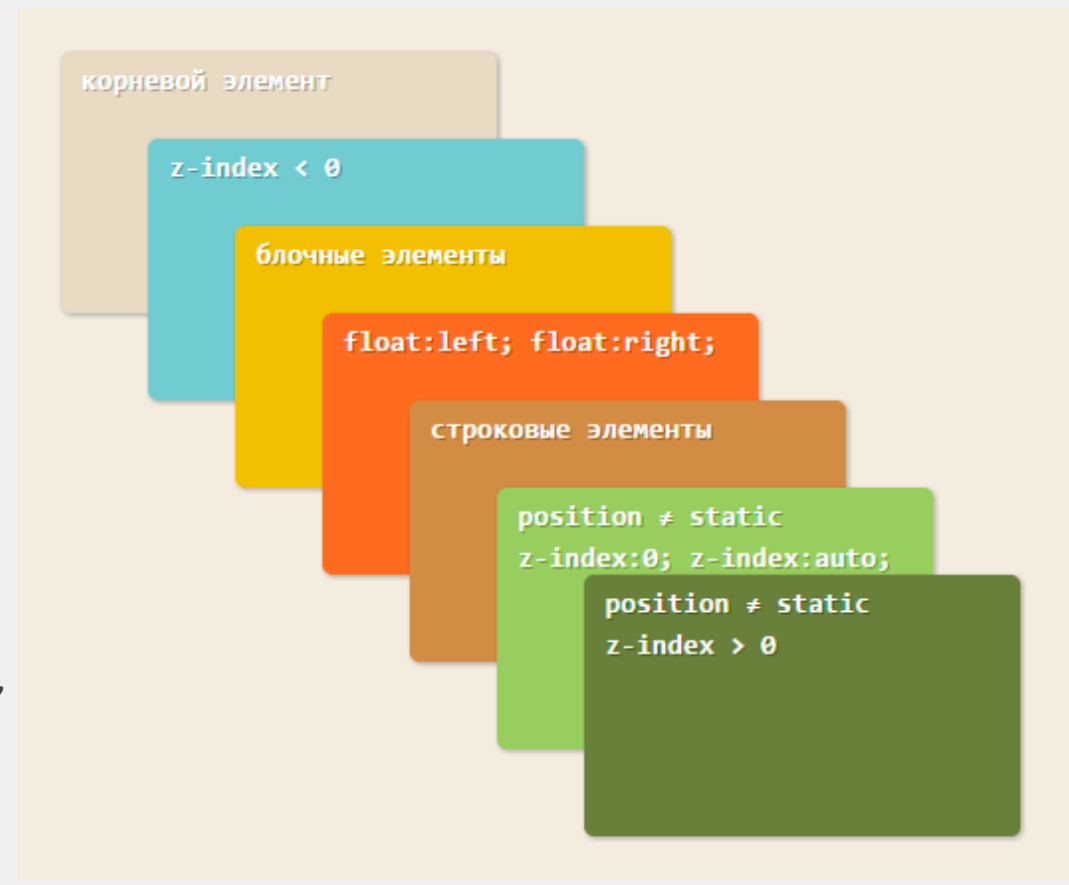
1. это корневой элемент (`<html>`);
2. элемент позиционирован абсолютно (`position: absolute`) или относительно (`position: relative`) со свойством `z-index`, значение которого не `auto`;
3. флекс-элемент со свойством `z-index`, значение которого не `auto` и чей родительский элемент имеет свойство `display: flex` или `display: inline-flex`;
4. элементу задано свойство `opacity` со значением меньше 1;
5. элементу задано свойство `transform` со значением не `none`;
6. элементу задано свойство `mix-blend-mode` со значением не `normal`;
7. элементу задано свойство `filter` со значением не `none`;
8. элементу задано свойство `isolation` со значением `isolate`;
9. элемент с `position: fixed`;
10. элементу задано свойство `will-change` или аналогичный атрибут;
11. элементу задано свойство `-webkit-overflow-scrolling` со значением `touch`.

**Элементы, не создающие собственный контекст наложения, используют родительский контекст.**

# Порядок наложения (от нижнего до верхнего слоя)

Внутри каждого контекста наложения существует свой порядок отрисовки элементов и их компонентов.

1. Фон (background) и рамки элемента (border);
2. Позиционированные элементы (включая потомков), у которых значение свойства `z-index` меньше 0;
3. Элементы без контекста наложения с `display: block`;
4. Элементы со свойством `float`;
5. Элементы с `display: inline`;
6. Элементы (включая потомков) с `z-index: auto` или `z-index: 0`, а также элементы с `opacity` меньше 1;
7. Элементы (включая потомков) с `z-index` больше 0.



Согласно порядку, элемент с отрицательным значением `z-index` никогда не будет ниже фона или рамки родительского элемента

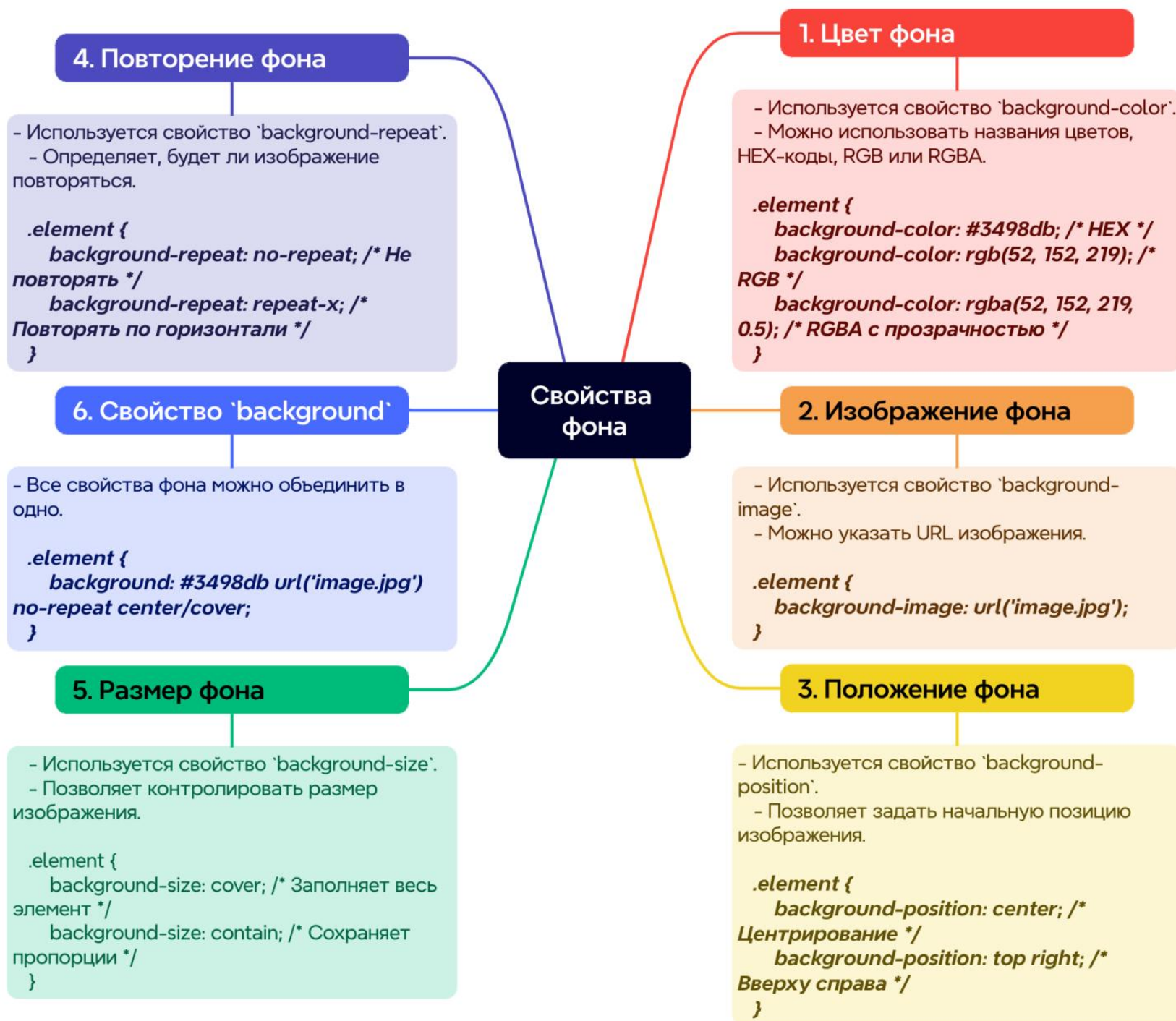
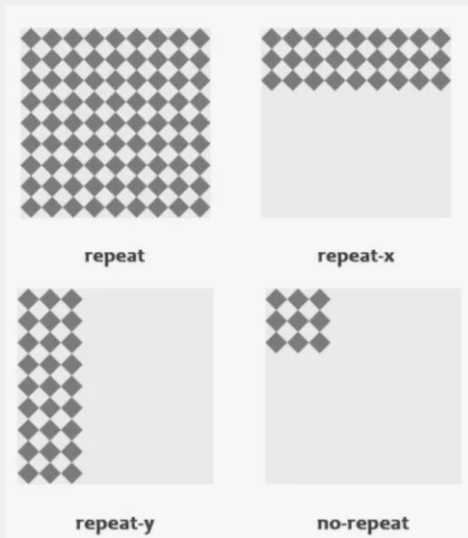
9

**ФОН, ГРАНИЦА, ЦВЕТ**

# Стилизация фона

Фон элемента можно стилизовать с помощью свойства **background**.

Это свойство позволяет задавать цвет, изображение, положение и другие параметры фона



Границы элементов задаются с помощью свойства "border".  
Это свойство позволяет определить стиль, ширину и цвет границы.

# Границы

|        |        |        |        |
|--------|--------|--------|--------|
| dotted | dotted | dotted | dotted |
| dashed | dashed | dashed | dashed |
| solid  | solid  | solid  | solid  |
| double | double | double | double |
| groove | groove | groove | groove |
| ridge  | ridge  | ridge  | ridge  |
| inset  | inset  | inset  | inset  |
| outset | outset | outset | outset |

## 4. Объединение границ

- все свойства границы можно объединить в одно.

```
.element {  
  border: 2px solid #e74c3c; /* Ширина,  
  стиль и цвет */  
}
```

## 5. \*Границы по сторонам

- можно задавать границы для каждой стороны отдельно.

```
.element {  
  border-top: 2px solid #e74c3c; /*  
  Верхняя граница */  
  border-right: 1px dashed #3498db; /*  
  Правая граница */  
  border-bottom: 3px dotted #2ecc71; /*  
  Нижняя граница */  
  border-left: 4px double #f1c40f; /*  
  Левая граница */  
}
```

## 3. Цвет границы

- используется свойство 'border-color'.

```
.element {  
  border-color: #e74c3c; /* HEX */  
  border-color: rgba(231, 76, 60, 0.5); /*  
  RGBA с прозрачностью */  
}
```

## Свойства границ

### 1. Ширина границы

- используется свойство 'border-width'.

```
.element {  
  border-width: 2px; /* Ширина границы */  
  border-width: 1px 2px 3px 4px; /*  
  Верхний, правый, нижний, левый */  
}
```

### 2. Стиль границы

- используется свойство 'border-style'.

```
.element {  
  border-style: solid; /* Сплошная линия */  
  border-style: dashed; /* Пунктирная  
  линия */  
  border-style: dotted; /* Точечная линия */  
  border-style: double; /* Двойная линия */  
  border-style: none; /* Без границы */  
}
```



# Цвета в вебе

**Цвет** — это один из важнейших аспектов веб-дизайна, который влияет на восприятие и функциональность сайта.

В CSS существует множество способов задания цвета, каждый из которых имеет свои особенности и применения.

Самый простой способ указания цвета – ключевые слова. Используются английские названия цветов. Самые популярные базовые цвета: чёрный/black, белый/white, красный/red, зелёный/green и синий/blue.

На самом деле ключевых слов для цветов очень много.

Нет особого смысла их все запоминать.

Всегда можно заглянуть в таблицу цветов

|                      |                   |                |                 |
|----------------------|-------------------|----------------|-----------------|
| IndianRed            | LightCoral        | Salmon         | DarkSalmon      |
| LightSalmon          | Crimson           | Red            | FireBrick       |
| DarkRed              | Pink              | LightPink      | HotPink         |
| DeepPink             | MediumVioletRed   | PaleVioletRed  | Coral           |
| Tomato               | OrangeRed         | DarkOrange     | Orange          |
| Gold                 | Yellow            | LightYellow    | LemonChiffon    |
| LightGoldenrodYellow | PapayaWhip        | Moccasin       | PeachPuff       |
| PaleGoldenrod        | Khaki             | DarkKhaki      | Lavender        |
| Thistle              | Plum              | Violet         | Orchid          |
| Fuchsia              | Magenta           | MediumOrchid   | MediumPurple    |
| RebeccaPurple        | BlueViolet        | DarkViolet     | DarkOrchid      |
| DarkMagenta          | Purple            | Indigo         | SlateBlue       |
| DarkSlateBlue        | MediumSlateBlue   | GreenYellow    | Chartreuse      |
| LawnGreen            | Lime              | LimeGreen      | PaleGreen       |
| LightGreen           | MediumSpringGreen | SpringGreen    | MediumSeaGreen  |
| SeaGreen             | ForestGreen       | Green          | DarkGreen       |
| YellowGreen          | OliveDrab         | Olive          | DarkOliveGreen  |
| MediumAquaMarine     | DarkSeaGreen      | LightSeaGreen  | DarkCyan        |
| Teal                 | Aqua              | Cyan           | LightCyan       |
| PaleTurquoise        | AquaMarine        | Turquoise      | MediumTurquoise |
| DarkTurquoise        | CadetBlue         | SteelBlue      | LightSteelBlue  |
| PowderBlue           | LightBlue         | SkyBlue        | LightSkyBlue    |
| DeepSkyBlue          | DodgerBlue        | CornflowerBlue | MediumSlateBlue |
| RoyalBlue            | Blue              | MediumBlue     | DarkBlue        |
| Navy                 | MidnightBlue      | Cornsilk       | BlanchedAlmond  |
| Bisque               | NavajoWhite       | Wheat          | BurlyWood       |
| Tan                  | RosyBrown         | SandyBrown     | Goldenrod       |
| DarkGoldenrod        | Peru              | Chocolate      | SaddleBrown     |
| Sienna               | Brown             | Maroon         | White           |
| Snow                 | HoneyDew          | MintCream      | Azure           |
| AliceBlue            | GhostWhite        | WhiteSmoke     | SeaShell        |
| Beige                | OldLace           | FloralWhite    | Ivory           |
| AntiqueWhite         | Linen             | LavenderBlush  | MistyRose       |
| Gainsboro            | LightGray         | Silver         | DarkGray        |
| Gray                 | DimGray           | LightSlateGray | SlateGray       |
| DarkSlateGray        | Black             |                |                 |

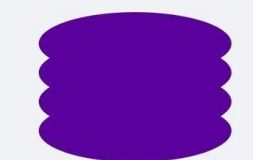
# Цвета в вебе. RGB

Для задания цвета используется функция `rgb()`, например, `rgb(0 63 255)` для синего. Каждое из трёх значений отвечает за отдельный канал RGB и может быть записано числом от 0 до 255 или в процентах. Для добавления прозрачности, после записи каналов нужно поставить слэш и записать нужное значение от 0 до 1 или в процентах, например `rgb(0 63 255 / 0.5)` для полупрозрачного синего.

Раньше синтаксис RGB отличался от современного. Для разделения каналов внутри функции нужно было использовать запятые `rgb(0, 63, 255)`, а для добавления прозрачности — специальную функцию `rgba()`, которая принимала последним параметром прозрачность цвета, например `rgba(255, 0, 0, 0.5)`

```
rgba(255, 255, 255, 1)
```

RED GREEN BLUE ALPHA



RGB (90, 0, 157)  
#5A009D



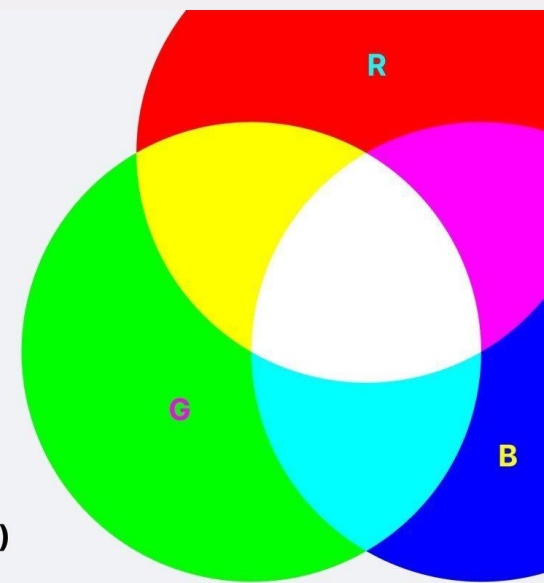
RGB (255, 223, 0)  
#FFDF00



RGB (0, 0, 0)  
#000000



RGB (255, 117, 195)  
#FF75C3





# Цвета в вебе. HEX

Шестнадцатеричный код цвета в цветовой модели RGB, который начинается с #, например, #ff0000. Сплошные цвета записываются в формате #RRGGBB или в сокращённом #RGB (если символы каждой группы одинаковые). Например, #090. Если нужно указать прозрачность, она добавляется в конце в HEX-формате #RRGGBBAA или #RGBA, например, #00990055 или #0905.

Раньше нельзя было задать цвет в нотации #RGBA и приходилось использовать функцию rgba(), но сегодня у способа неплохая кроссбраузерность.

Мало кто сходу сможет рассчитать 50% в этом формате, поэтому для указания прозрачности цвета чаще всего используют функцию rgb(), где прозрачность можно задать в дробях или процентах.

|    | A         | B       | C       | D       | E       | F       | G       | H       | I       | J       | K       |
|----|-----------|---------|---------|---------|---------|---------|---------|---------|---------|---------|---------|
| 1  | Color     |         |         |         |         |         |         |         |         |         |         |
| 2  | HEX value | #000000 | #434343 | #666666 | #999999 | #b7b7b7 | #cccccc | #d9d9d9 | #efefef | #f3f3f3 | #ffffff |
| 3  | Color     |         |         |         |         |         |         |         |         |         |         |
| 4  | HEX value | #980000 | #ff0000 | #ff9900 | #ffff00 | #00ff00 | #00ffff | #4a86e8 | #0000ff | #9900ff | #ff00ff |
| 5  | Color     |         |         |         |         |         |         |         |         |         |         |
| 6  | HEX value | #e6b8af | #f4cccc | #fce5cd | #fff2cc | #d9ead3 | #d0e0e3 | #c9daf8 | #cfe2f3 | #d9d2e9 | #ead1dc |
| 7  | Color     |         |         |         |         |         |         |         |         |         |         |
| 8  | HEX value | #dd7e6b | #ea9999 | #f9cb9c | #ffe599 | #b6d7a8 | #a2c4c9 | #a4c2f4 | #9fc5e8 | #b4a7d6 | #d5a6bd |
| 9  | Color     |         |         |         |         |         |         |         |         |         |         |
| 10 | HEX value | #cc4125 | #e06666 | #f6b26b | #ffd966 | #93c47d | #76a5af | #6d9eeb | #6fa8dc | #8e7cc3 | #c27ba0 |
| 11 | Color     |         |         |         |         |         |         |         |         |         |         |
| 12 | HEX value | #a61c00 | #cc0000 | #e69138 | #f1c232 | #6aa84f | #45818e | #3c78d8 | #3d85c6 | #674ea7 | #a64d79 |
| 13 | Color     |         |         |         |         |         |         |         |         |         |         |
| 14 | HEX value | #85200c | #990000 | #b45f06 | #bf9000 | #38761d | #134f5c | #1155cc | #0b5394 | #351c75 | #741b47 |
| 15 | Color     |         |         |         |         |         |         |         |         |         |         |
| 16 | HEX value | #5b0f00 | #660000 | #783f04 | #7f6000 | #274e13 | #0c343d | #1c4587 | #073763 | #20124d | #4c1130 |

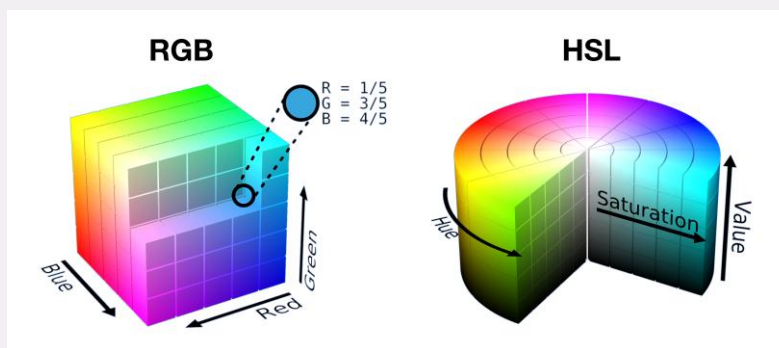
# Цвета в вебе. HSL

HSL описывает те же цвета, что и RGB, но иначе: H — Hue (оттенок), S — Saturation (насыщенность), L — Lightness (светлота). Например, `hsl(120 100% 50%)` для зелёного. Первое значение оттенка задаётся в градусах и его можно записать просто как 120 (как чаще всего и делают) или с указанием единиц поворота: `deg`, `turn`, `grad`, `rad`.

Доступны значения от 0 до 360, если указываете поворот в градусах. Отрицательные значения не допускаются. Второе и третье значение указываются в процентах. Прозрачность добавляется так же, как в `rgb`, с помощью слэша со значением, например `hsl(120 100% 50% / 0.5)` полупрозрачный зелёный.

Синтаксис `hsl()` отличался от современного точно так же, как `rgb()`: нужны были запятые и специальная функция `hsla()` для задания прозрачности цвета.

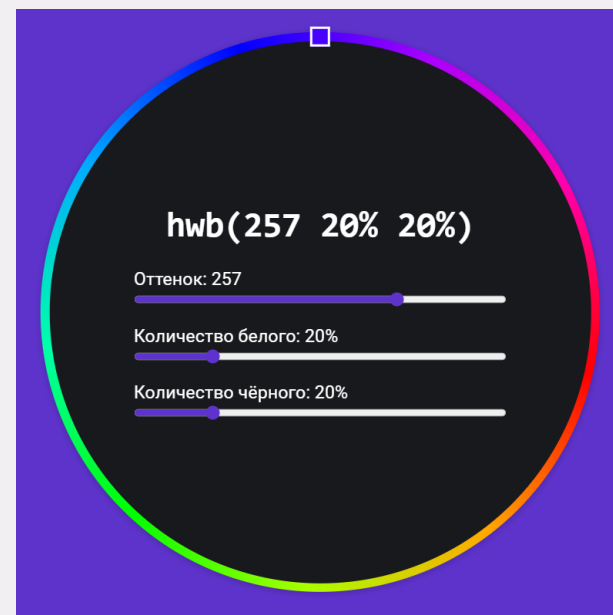
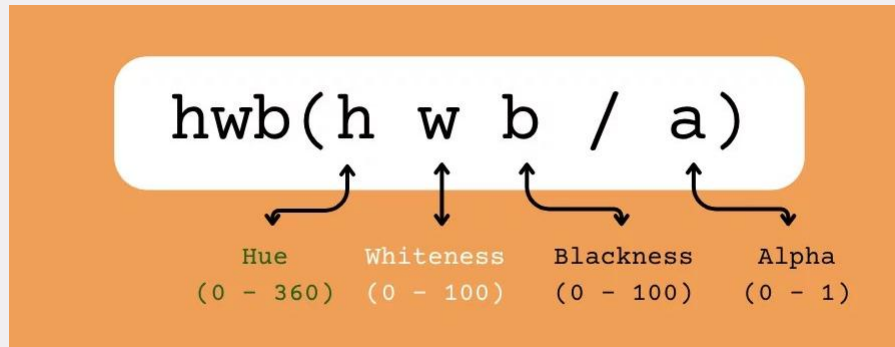
Используйте старый синтаксис для лучшей совместимости.



# Цвета в вебе. HWB

Н — hue (оттенок), W — whiteness (количество белого), B — blackness (количество чёрного). Первое значение оттенка задаётся в градусах и его можно записать просто как 120 (как чаще всего и делают) или с указанием единиц поворота: deg, turn, grad, rad. Доступны значения от 0 до 360, если указываете поворот в градусах. Второе и третье значения указываются в процентах. Допустимо указывать прозрачность цвета четвёртым параметром через слэш.

Если второй и третий параметры в сумме дают 100%, то они нормализуются до 100% автоматически



# Ключевые слова

## currentColor

За ключевым словом **currentColor** скрывается цвет, указанный в свойстве **color** элемента. Удобно указывать это слово в качестве значения других свойств, чтобы многократно не указывать один и тот же цвет.

Заголовок

```
h1 {  
  color: ■ #6e4aff;  
  border: 1px solid currentColor;  
}
```

Если мы захотим менять цвет текста и рамки по наведению курсора, то нам достаточно будет изменить значение одного свойства, а цвет рамки подстроится за счёт **currentColor**

Заголовок

Заголовок

```
h1:hover {  
  color: ■ #09ff00;  
}
```

## transparent

Ключевое слово **transparent** задаёт прозрачный цвет текста.

Технически это равносильно записи любого цвета с нулевой прозрачностью **rgb(0 0 0 / 0)**, но бывают случаи, когда просто прозрачность и прозрачность цвета могут работать иначе, например, в градиентах.

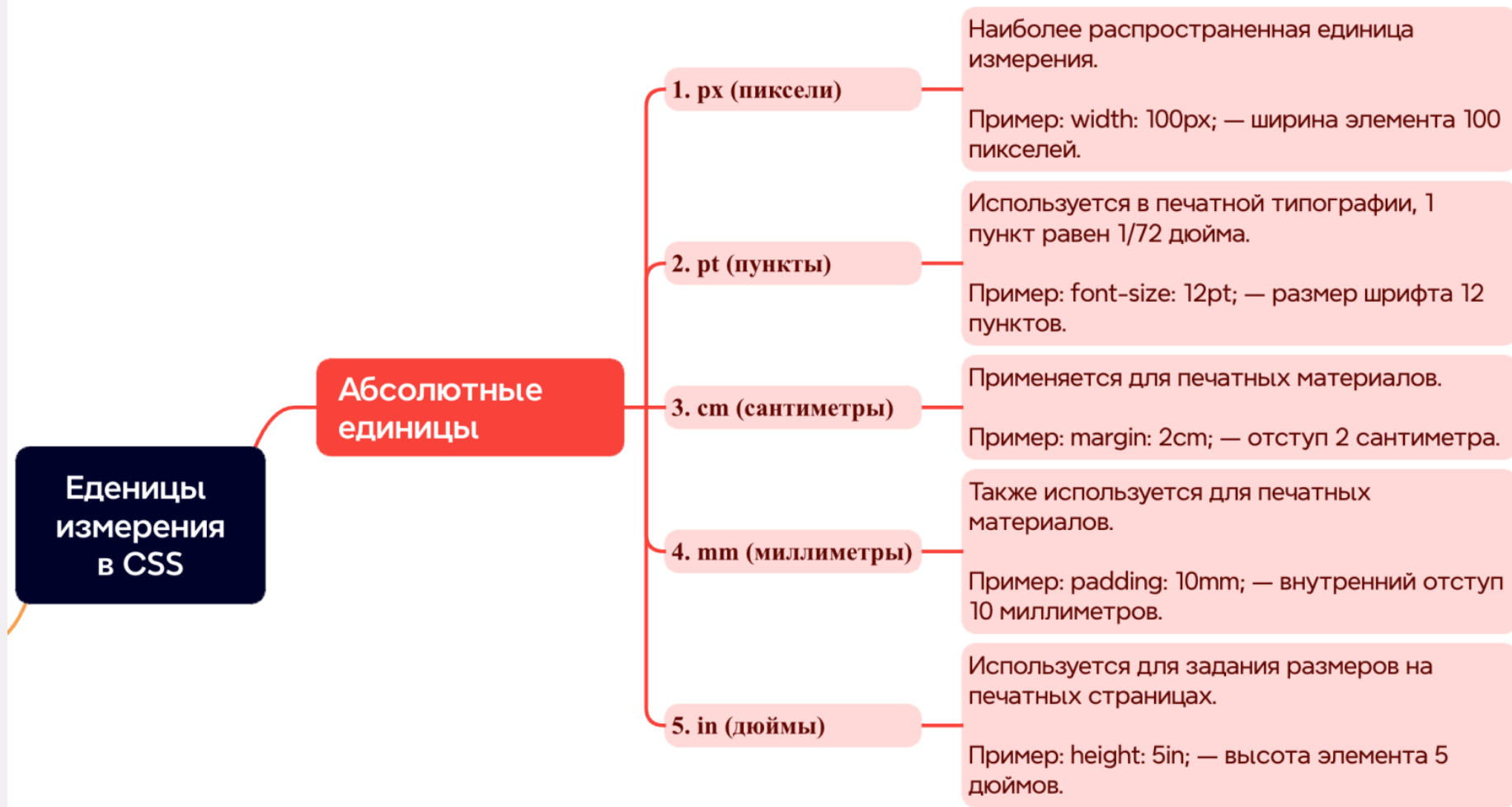
10

# ЕДИНИЦЫ ИЗМЕРЕНИЯ

# Единицы измерения

## Абсолютные единицы

ни от чего не зависят  
и привязаны к физическим  
единицам измерения:  
дюймам или сантиметрам



В CSS существует несколько единиц измерения, которые используются для задания размеров, расстояний и др. параметров

# Единицы измерения

## Относительные единицы

Используются для задания размера или расстояния относительно чего-либо.

Например, `vh` считается относительно высоты вьюпорта (области просмотра страницы в окне браузера)

Относится к размеру шрифта родительского элемента. `1em` равен текущему размеру шрифта.

Пример: `font-size: 2em;` — размер шрифта в два раза больше размера родительского элемента.

1. `em`

Относится к размеру шрифта корневого элемента (`<html>`). Это делает его более предсказуемым по сравнению с `em`.

Пример: `font-size: 1.5rem;` — размер шрифта в 1.5 раза больше размера шрифта корневого элемента.

2. `rem (root em)`

Задаёт размеры относительно родительского элемента.

Пример: `width: 50%;` — ширина элемента составляет 50% от ширины родительского элемента.

3. `% (проценты)`

Относится к ширине области просмотра. `1vw` равен 1% от ширины области просмотра.

Пример: `width: 50vw;` — ширина элемента составляет 50% от ширины окна браузера.

4. `vw (viewport width)`

Относится к высоте области просмотра. `1vh` равен 1% от высоты области просмотра.

Пример: `height: 100vh;` — высота элемента равна высоте окна браузера.

5. `vh (viewport height)`

`vmín` — минимальное значение между шириной и высотой области просмотра, `vmáx` — максимальное значение.

Пример: `font-size: 5vmín;` — размер шрифта равен 5% от меньшего из значений ширины или высоты окна браузера.

6. `vmín` и `vmáx`

Относительные единицы

Единицы измерения в CSS

# Единицы измерения углов

Используются, как правило, для задания угла поворота элемента, направления линейного градиента или угла поворота конического градиента.

**deg** — градусы угла. В CSS 0deg соответствует пересечению окружности с осью y, а градусы отсчитываются по часовой стрелке

**grad** — градианы. 1/100 от прямого угла ( $0.9^\circ$ ).  
Полная окружность = 400 градиан.

**rad** — радианы. Окружность =  $2\pi$  ( $\sim 6.2832$ ) радиан.

**turn** — оборот. Полная окружность соответствует одному обороту (1turn). Поворот элемента на 90 градусов соответствует 0.25turn.

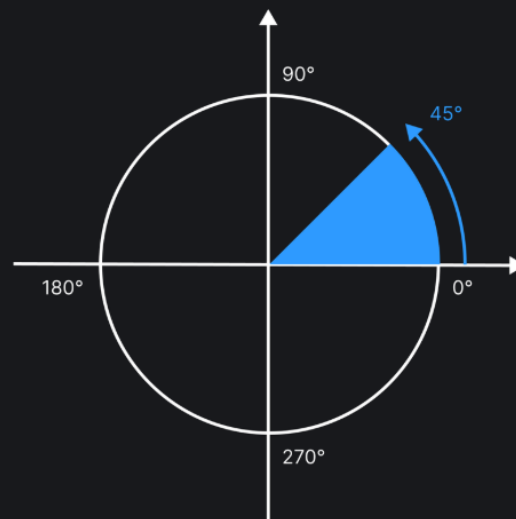
# Единицы измерения времени

Используются для задания длительности анимации или перехода в свойствах **animation** и **transition**.

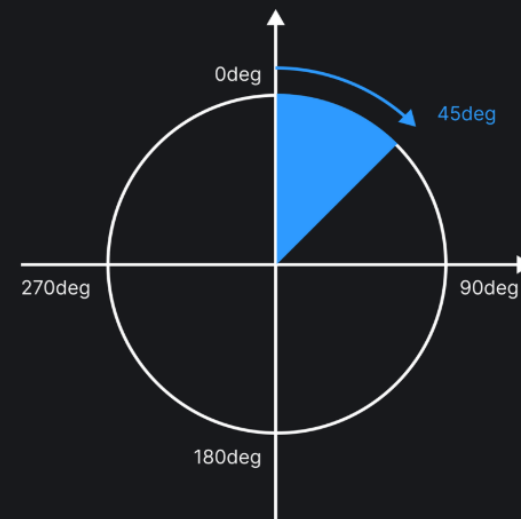
**ms** — миллисекунда.

**s** — секунда.  $1s = 1000ms$

В тригонометрии



В CSS





# Единицы, связанные с разрешением экрана

Используются для указания плотности пикселей (разрешения экрана) в медиавыражениях.

**dpi** — указывает плотность пикселей в точках на дюйм (dots per inch). Обычные экраны имеют плотность пикселей 72 или 96dpi, но при печати это разрешение гораздо выше (300 и более точек на дюйм).  $1\text{dpi} = 0.39\text{dpcm}$ .

**dpcm** — плотность пикселей в точках на сантиметр.  $1\text{dpcm} = 2.54\text{dpi}$ .

**dppx, x** — описывает плотность в точках на px. Поскольку соотношение in и px равно 1/96,  $1\text{dppx} = 96\text{dpi}$

# Проценты

100%

20rem

25vw

Используются, чтобы указать, что значение представляет собой долю от другой величины.

Исходное значение, от которого берётся часть, может относиться как к самому элементу, так и к его предку. Всё зависит от того, для какого свойства мы применяем проценты.

Например, при указании `width: 50%` ширина элемента будет высчитана как 50% (половина) от ширины родителя. Но если мы укажем в процентах положение фона **background-position-x: 30%**, это значение будет высчитано относительно ширины самого элемента.

11

# ПСЕВДОЭЛЕМЕНТЫ

# Псевдоэлементы

**Псевдоэлементы** — это элементы, которых не существует в HTML-разметке. Они создаются и позиционируются исключительно при помощи CSS. Чаще всего используются для создания различных декоративных элементов (которые не несут содержательного смысла).

## ::before и ::after

Два самых часто встречающихся псевдоэлемента. Очень похожи. Единственная разница заключается в том, что **::before** по умолчанию располагается перед содержимым элемента, для которого задаётся, а **::after** — после.

Для обоих элементов обязательным является свойство **content** — содержимое псевдоэлемента. С его помощью можно, например, вставить какое-нибудь слово до или после текста. Без свойства **content** псевдоэлемент не отобразится

```
<span class="incoming">друг</span>
```

```
.incoming::before {  
  content: "Привет, ";  
}  
  
.incoming::after {  
  content: ", ты отлично выглядишь!";  
}
```

Привет, друг, ты отлично выглядишь!

## ::first-letter

При помощи этого псевдоэлемента можно выбрать первую букву в строке или абзаце текста.

Это позволяет создать буквицу — большую декоративную букву в начале текста.

```
<p class="text">Белый снег, пушистый <br>  
  В воздухе кружится <br>  
  И на землю тихо <br>  
  Падает, ложится.</p>
```

```
.text::first-letter {  
  font-size: 52px;  
  color: #F498AD;  
}
```

Белый снег, пушистый  
В воздухе кружится  
И на землю тихо  
Падает, ложится.

## ::first-line

Этот псевдоэлемент выбирает первую строку текста. Обратите внимание, что он сработает только для блочных элементов.

```
.text::first-line {  
  background-color: #F498AD;  
}
```

Белый снег, пушистый  
В воздухе кружится  
И на землю тихо  
Падает, ложится.

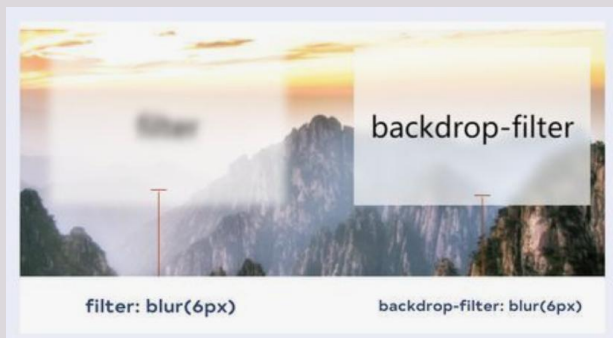
## ::selection

С помощью псевдоэлемента **::selection** можно управлять стилем текста, который пользователь выделяет при помощи мыши.

```
::selection {  
  background-color: #98f4ba;  
}
```

Белый снег, пушистый  
В воздухе кружится  
И на землю тихо  
Падает, ложится.

**::backdrop** отвечает за подложку, появляющуюся под всплывающими на верхний уровень элементами.



## ::backdrop

## ::placeholder

Позволяет стилизовать подсказку, которая выводится в поле ввода текста (<input>).

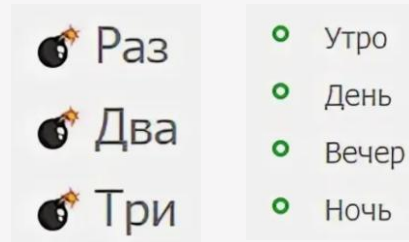
Текст подсказки задаётся при помощи атрибута **placeholder** у тега <input>

Field label

Input Placeholder

This is a short description.

**::marker** отвечает за маркерное поле. В нём находятся, например, маркеры списка



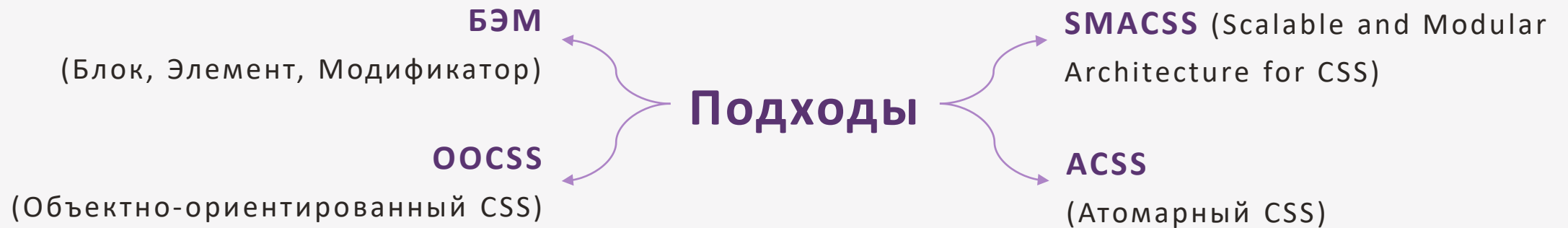
## ::marker

12

## ПОДХОДЫ К СОЗДАНИЮ КЛАССОВ, ИЛИ АРХИТЕКТУРА CSS

Существует несколько подходов к созданию классов в CSS, каждый из которых имеет свои преимущества и недостатки.

## Подходы к созданию классов CSS



В крупных проектах зачастую наблюдается структурная организация JavaScript-кода при полном отсутствии архитектуры в CSS. Разработчики сталкиваются с хаотичным набором правил, множественными переопределениями и некорректным использованием `!important`. Для решения этой проблемы были разработаны архитектурные методологии.

Пренебрежение архитектурой CSS недопустимо, учитывая его фундаментальную роль.

Внедрение принципов на начальном этапе помогает минимизировать дальнейшие трудности.

# БЭМ

BEM = Block Element Modifier (БЭМ = Блок Элемент Модификатор). Подход предлагает единообразное соглашение по именованию CSS-классов.

Рекомендуемая схема именования:

**имя-блока\_\_имя-элемента--имя-модификатора.**

- **имя-блока** определяет стилизуемый компонент верхнего уровня, например **photo**.
- **\_\_имя-элемента** обозначает элемент внутри этого блока. Пример **photo\_\_image** или **photo\_\_caption**.
- **--имя-модификатора** описывает модификатор для блока или элемента, например **photo--large** или **photo\_\_image--black-and-white**.

Для более глубокого ознакомления с методологией рекомендуется изучить <https://css-tricks.com/bem-101/>

# OOCSS

**OOCSS** – методология, основанная на принципах абстракции для разделения структурных и визуальных стилей, что позволяет устранить дублирование в коде.

Вместо прямого назначения свойств каждому компоненту

OOCSS предлагает декомпозицию

Теперь **.primary** может быть переиспользован для других элементов. Этот подход соответствует принципу DRY (Don't Repeat Yourself), поддерживает компактность.

```
.button {  
  padding: 10px 16px;  
  color: blue;  
}  
  
.h1 {  
  font-size: 2em;  
  color: blue;  
}
```

```
.button {  
  padding: 10px 16px;  
}  
  
.h1 {  
  font-size: 2em;  
}
```

```
.primary {  
  color: blue;  
}
```

## An Introduction To Object Oriented CSS

<https://www.smashingmagazine.com/2011/12/an-introduction-to-object-oriented-css-oocss/>



# SMACSS

– это архитектурный подход, организующий стили в пять строгих категорий.

- **Базовые стили (Base)** определяют свойства по умолчанию для всех HTML-элементов.
- **Стили макета (Layout)** описывают структурные компоненты: сетки, шапку, подвал.
- **Стили модулей (Module)** содержат правила для повторно используемых компонентов интерфейса.
- **Стили состояний (State)** фиксируют визуальные изменения элементов (например, is-active, is-hidden).
- **Тематические стили (Theme)** определяют брендинг и визуальную идентичность, выходящие за рамки базовых настроек.

Для ознакомления с методологией рекомендуется официальное руководство на <https://smacss.com/>

# ACSS

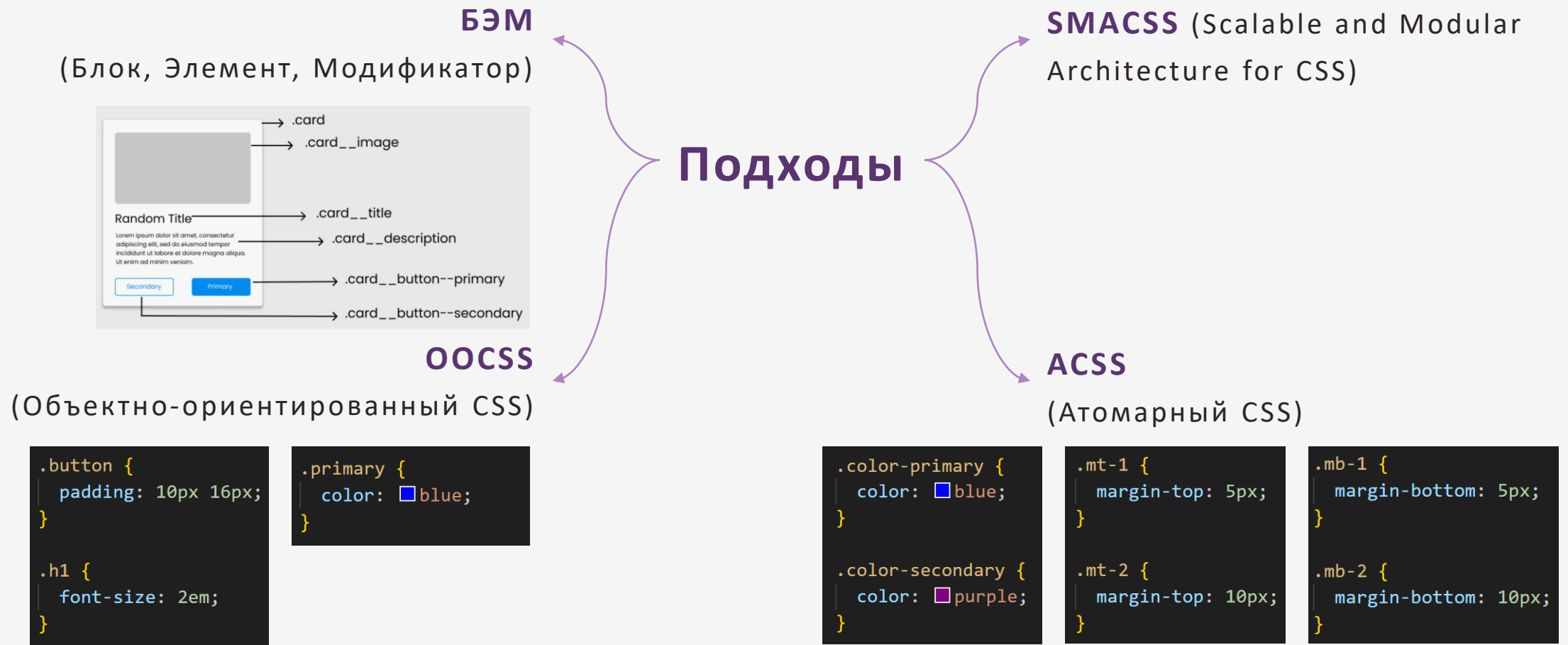
– утилитарный подход, основанный на создании множества атомарных CSS-классов, каждый из которых отвечает за одно конкретное свойство. В отличие от OOCSS, где абстракции более высокоуровневые, ACSS доводит принцип разделения стилей до минимального уровня.

Ключевое отличие ACSS от OOCSS – явное отражение CSS-свойств в именах классов (например, .color-primary вместо .primary), что обеспечивает прямую связь между классом и применяемым стилевым правилом

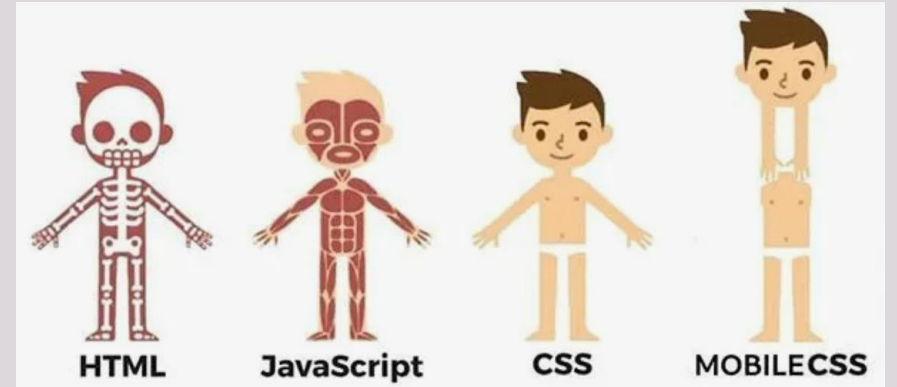
```
.color-primary {  
  color: blue;  
}  
  
.color-secondary {  
  color: purple;  
}  
  
.mt-1 {  
  margin-top: 5px;  
}  
  
.mt-2 {  
  margin-top: 10px;  
}  
  
.mb-1 {  
  margin-bottom: 5px;  
}  
  
.mb-2 {  
  margin-bottom: 10px;  
}
```

Существует несколько подходов к созданию классов в CSS, каждый из которых имеет свои преимущества и недостатки.

# Подходы к созданию классов CSS



**СПАСИБО ЗА ВНИМАНИЕ!**



26 ноября 2025 года